Module Code & Module Title

CS5003NI Data Structures and Specialist Programming

30% Individual Coursework 2

Submission: Final Submission.

Academic Semester: AY 2025/2026

Credit: 30 credit year long module

Student Name: Balram Ray

Project Title: Gokyo Bistro Restaurant Management System.

London Met ID: 24046669

College ID: np01ai4a240044

Assignment Due Date: 17/05/2026

Assignment Submission Date: 14/05/2026

Submitted To: Mr. Sudip Dahal

GitHub Link	https://github.com/balramyRay/Gokyo-Bistro-.git

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Coursework Final Submission

Islington College,Nepal

Document Details

Submission ID **trn:oid:::3618:138900444**

Submission Date

May 14, 2026, 10:26 AM GMT+5:45

Download Date

May 14, 2026, 10:28 AM GMT+5:45

File Name

24046669 Balram Ray.pdf

File Size

3.6 MB

84 Pages

15,100 Words

83,803 Characters

12% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Small Matches (less than 8 words)

Exclusions

- 1 Excluded Source

Match Groups

86 Not Cited or Quoted 11%
Matches with neither in-text citation nor quotation marks

2 Missing Quotations 0%
Matches that are still very similar to source material

12 Missing Citation 1%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

8% Internet sources

4% Publications

12% Submitted works (Student Papers)

Match Groups

- **86** Not Cited or Quoted 11%
  Matches with neither in-text citation nor quotation marks
- **2** Missing Quotations 0%
  Matches that are still very similar to source material
- **12** Missing Citation 1%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

8% Internet sources
4% Publications
12% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	
National School of Business Management NSBM, Sri Lanka on 2025-05-10		1%
2	Submitted works	
National School of Business Management NSBM, Sri Lanka on 2025-05-11		<1%
3	Submitted works	
Illinois Institute of Technology on 2024-12-09		<1%
4	Submitted works	
University of Northampton on 2025-04-26		<1%
5	Submitted works	
Southampton Solent University on 2024-11-29		<1%
6	Internet	
www.coursehero.com		<1%
7	Submitted works	
University of Northampton on 2025-05-19		<1%
8	Submitted works	
University of Bedfordshire on 2025-12-10		<1%
9	Submitted works	
National School of Business Management NSBM, Sri Lanka on 2025-05-10		<1%
10	Submitted works	
United World College of South East Asia on 2025-09-10		<1%

Table of Contents

List Of Tables

List Of Figures

1.Introduction

1.1Title

Gokyo Bistro Restaurant Management System

1.2 Purpose

The purpose of this project is to create a web application based system for Gokyo Bistro restaurant to help restaurant manage their daily operations online. Currently the restaurant handles table reservations and food orders manually through phone calls and paper records. This causes problems like double booking of tables, lost order details and difficulty in tracking customer information.

This system will allow customers to book tables online, order food for delivery, and view the menu from their phone or computer. It will also help the restaurant owner to manage menu items, view all reservations and orders, post discount offers and see revenue reports. The goal is to make the restaurant work faster, easier and more organized.

1.3Audience

The intended audience for this application includes two main groups:

Restaurant Customers (Members) – People who want to book a table at Gokyo Bistro or order food for home delivery. They will use the system to create an account, login, book tables, place orders, and view their reservation, order history and manage their profile.

Restaurant Manager (Admin) – The person who runs the restaurant. They will use the system to add or remove menu items, see all customer reservations and orders and manage the member account.

1.4 Aims and Objectives

Main Aim:

To build a complete online management system for Gokyo Bistro restaurant that handles table reservations, food ordering, menu management and customer accounts.

Specific Objectives:

- To create a secure login and registration system where customers can create accounts and login.
- To build a table booking feature that lets customers choose date, time and table number.
- To develop a food ordering feature for home delivery with address collection.
- To create an admin panel where the restaurant manager can add, edit, and delete menu items.
- To allow admin to view all customer reservations and orders in one place.
- To let admin manage customers account.
- To add a menu page with search functionality so customers can find food items easily.
- To include an about page and contact page for restaurant information.
- To make the website work properly on both computers and mobile phones using responsive design.

2.Problems with the Current Manual System

Currently, Gokyo Bistro restaurant handles everything through phone calls, paper notebooks and walk in customers. This old way of working has many problems.

A) Double Booking of Tables

When customers call to book a table, the staff writes the booking in a notebook. Sometimes two different staff members make bookings for the same table at the same time. The customer arrives at the restaurant and finds their table already taken by someone else. This makes customers angry and unhappy.

How my system solves this: The system checks if a table is already booked at that date and time. If the table is not free then the customer cannot book it. This stops double booking completely.

B) Lost or Missing Order Details

The staff writes food orders on paper slips. These papers can get lost, torn or mixed up with other orders. The kitchen may not get the order or they may cook the wrong food. Customers wait for a long time and sometimes never get their food.

How my system solves this: All orders are saved in a database. The kitchen can see the order on a screen and nothing gets lost. The customer can also see their order status online.

C) No Record of Customer History

The restaurant has no way to know which customers came before, what they ordered or how many times they visited. When a regular customer calls, the staff cannot see their past orders or preferences. They have to ask the same questions every time.

How my system solves this: Every customer has an account. The system saves all their past reservations and orders. When a customer logs in they can see their own history. The admin can also see customer activity.

D) Difficulty Managing Menu Items

When the restaurant wants to add a new dish or remove an old one, they have to print new paper menus or tell customers verbally. This is slow and costly. If the price of a dish changes, customers may not know until they get the bill.

How my system solves this: The admin can add, edit, or delete any menu item from the admin panel. Changes appear immediately on the website. Customers always see the correct menu and prices.

E) No Online Ordering for Delivery

Customers who want food delivered to their home must call the restaurant. The staff takes the order over the phone. This can lead to mistakes in address, wrong items or wrong quantity. Also during busy hours, the phone lines are always busy and customers cannot get through.

How my system solves this: Customers can place orders directly from the website. They select the food items, choose quantity and type their delivery address. The order goes directly to the system. No phone calls needed.

F) No Way to Track Offers and Discounts

When the restaurant runs a special offer they put a sign outside or tell customers when they come in. Customers who are at home do not know about the offer. The restaurant loses potential sales.

How my system solves this: The admin can post discount offers on the website. All customers can see the offers when they visit the site. The system also shows the offer price next to the original price.

G) No Revenue Reports

The owner has no easy way to know how much money the restaurant made from online orders. They have to add up paper bills manually or check bank statements. This takes a lot of time and can have mistakes.

How my system solves this: The admin can see a report showing total number of orders and total revenue. The system calculates everything automatically.

3.Wireframe

wireframe is like a simple sketch of a website or app before fully designed. Think of it as the blueprint of a house, it shows the structure but not the paint or furniture.

3.1 Wireframe 1: Home page Wireframe

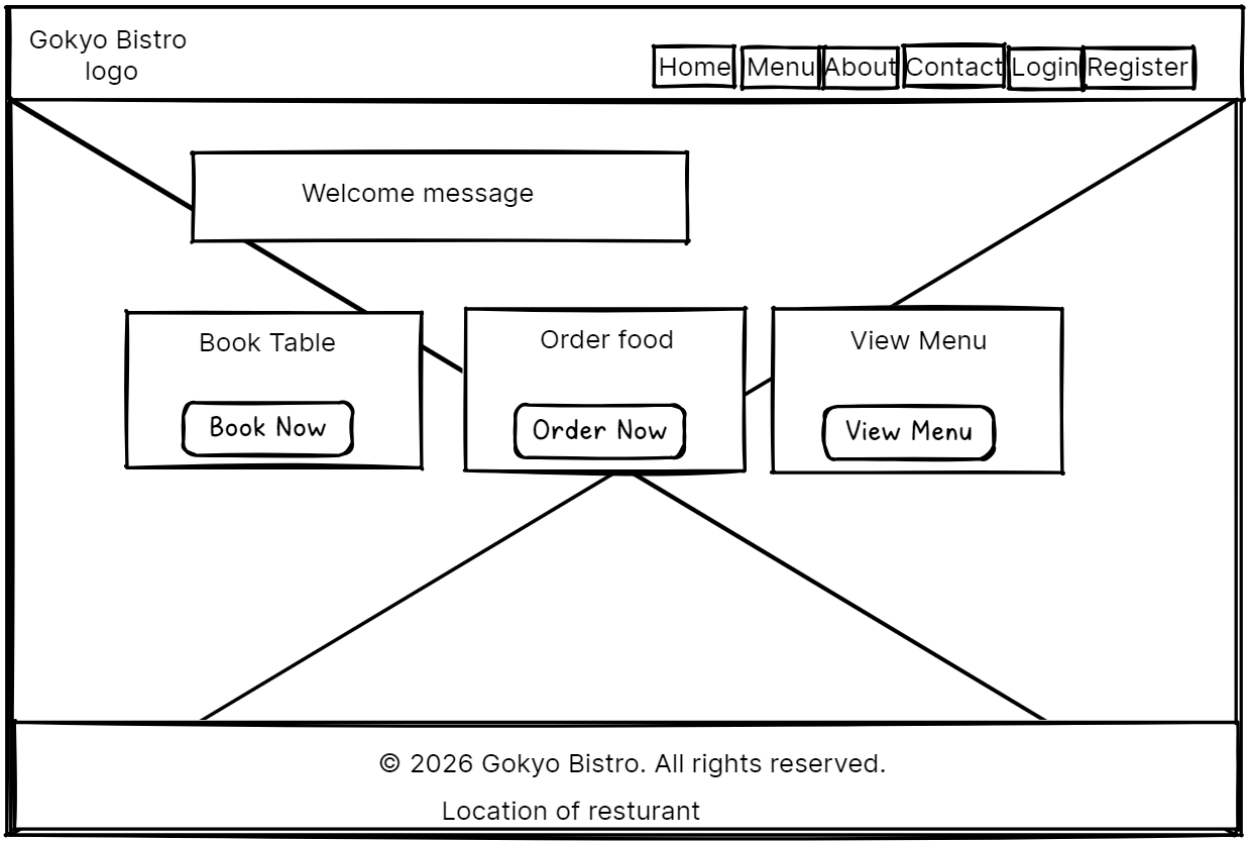


Figure 1:Home Page wireframe

This home page wire frame shows the first screen page will be open when the Gokyo bistro restaurant management is open. In this page at upper (i.e. header) having logo in place of left side currently showing Gokyo bistro logo, in the right side of the header having navigation link for home page, Menu Page, About Page, Contact page, and Register page.

In the middle having welcome message to visitor and having boxes showing Book table, Order food, View Menu with respective button for perform action. But without login Book table and Order food action cannot be performed, visitor only can view menu available.

In the footer having right reserved and location of restaurant.

3.2 Wireframe 2: Menu Page Wireframe

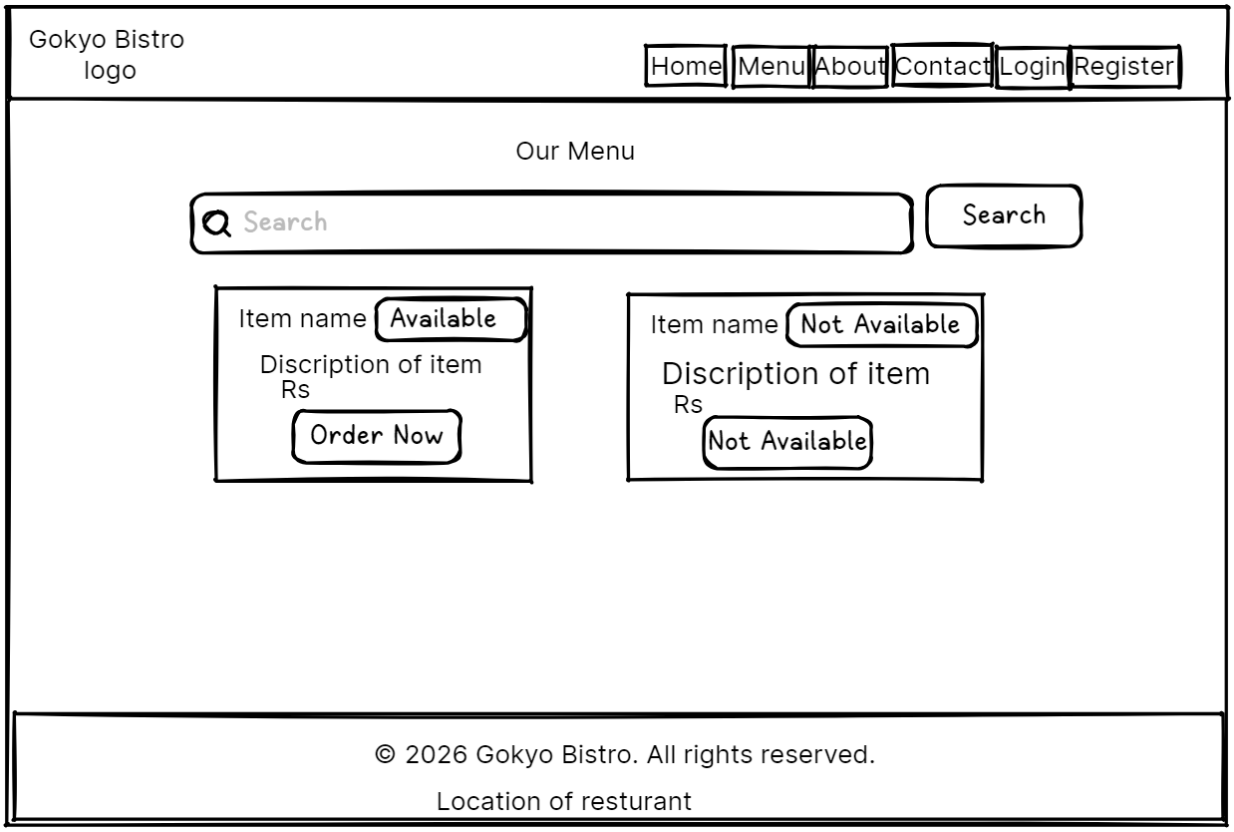


Figure 2:Menu Page wireframe

This Menu page will be open after clicking navigation link in header as Menu, it will be open to visitors without required login. This page having same header and footer like Home page.

In the middle having Our Menu having search box with search button to search menu items which display the match items from the menu. Below the search box displaying items with their name with name and availability and not availability status where if item is available can only be available for order , if not then cannot be order simple not available for order as seen in the wireframe.

But without login order operation cannot be performed by visitors. When click to the order button there is requirement for member through login.

3.3 Wireframe 3: About Page Wireframe

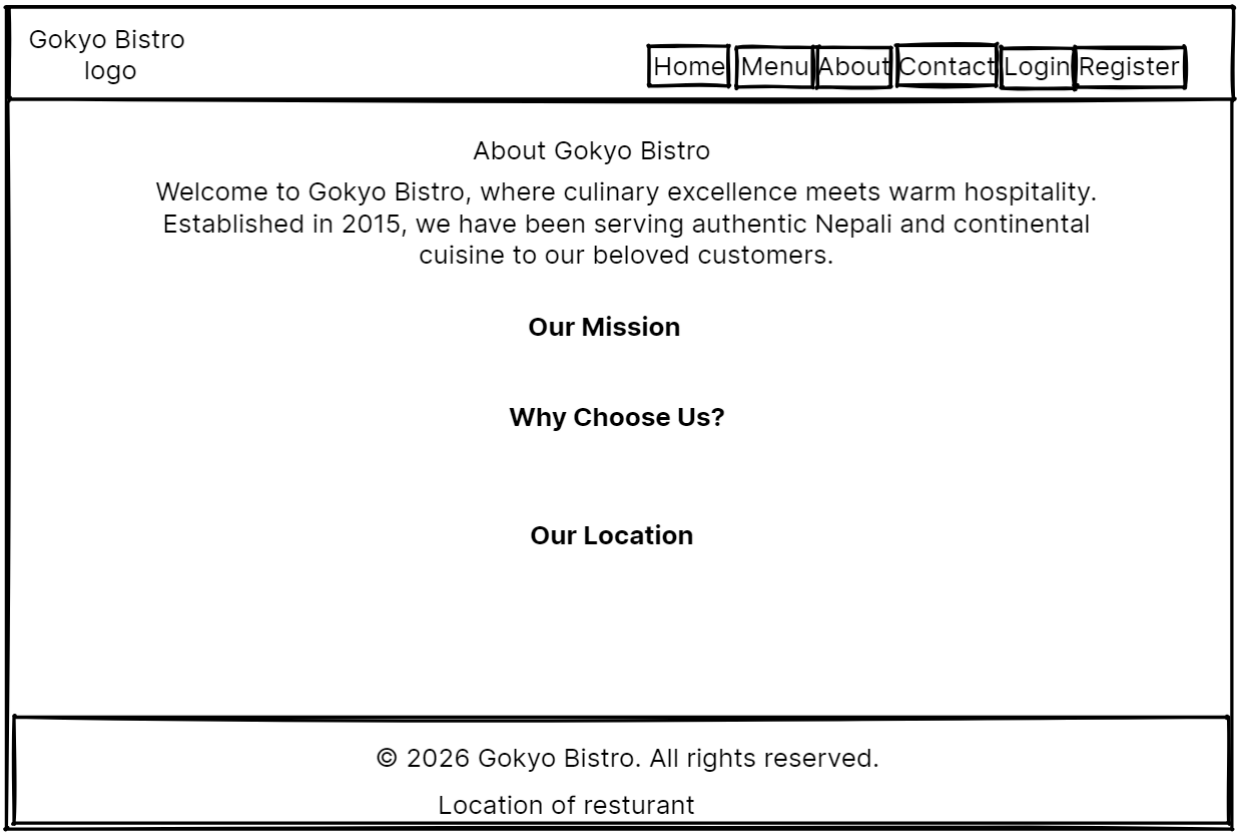


Figure 3: About Page Wireframe

This wireframe helps me plan how the About page will look before I write the actual code. It shows where everything will be placed on the screen.

Having same header and footer in the home page. In the middle having main content heading shows about this page as “About Gokyo Bistro”. This heading gives short introduction telling that restaurant started in 2015 and serves Nepali and continental food

Also having heading to give our mission information, why choose us information and location of the restaurant.

3.4 Wireframe 4: Contact Page Wireframe

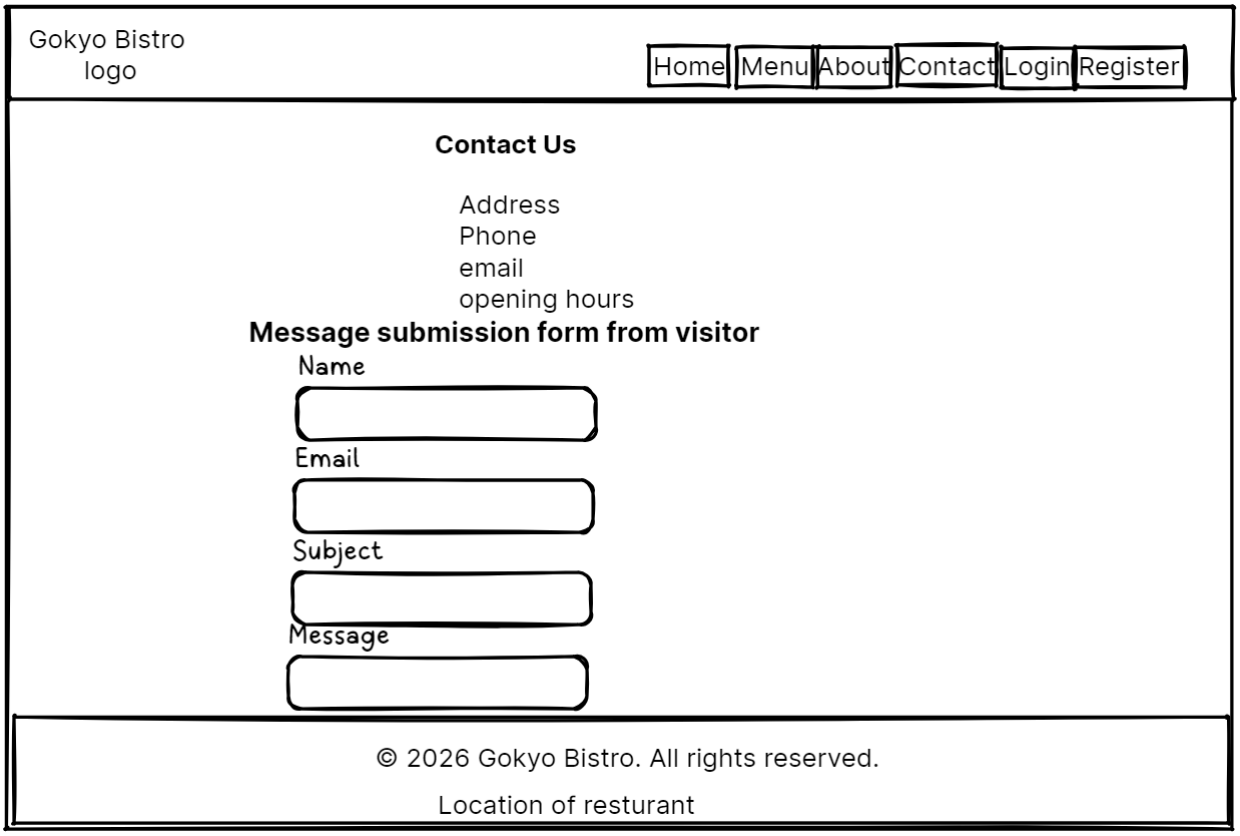


Figure 4:Contact Page wireframe

This page is important for your Gokyo Bistro website because it helps customers reach us easily.

Through this page customers can ask questions, can give feedback, Business inquiries. Also this page make website more professionals also give more information phone, email, opening hours.

In this website having same header and footer view like home page. In the middle having submission form through take information from customers.

3.5 Wireframe 5: Login Page Wireframe

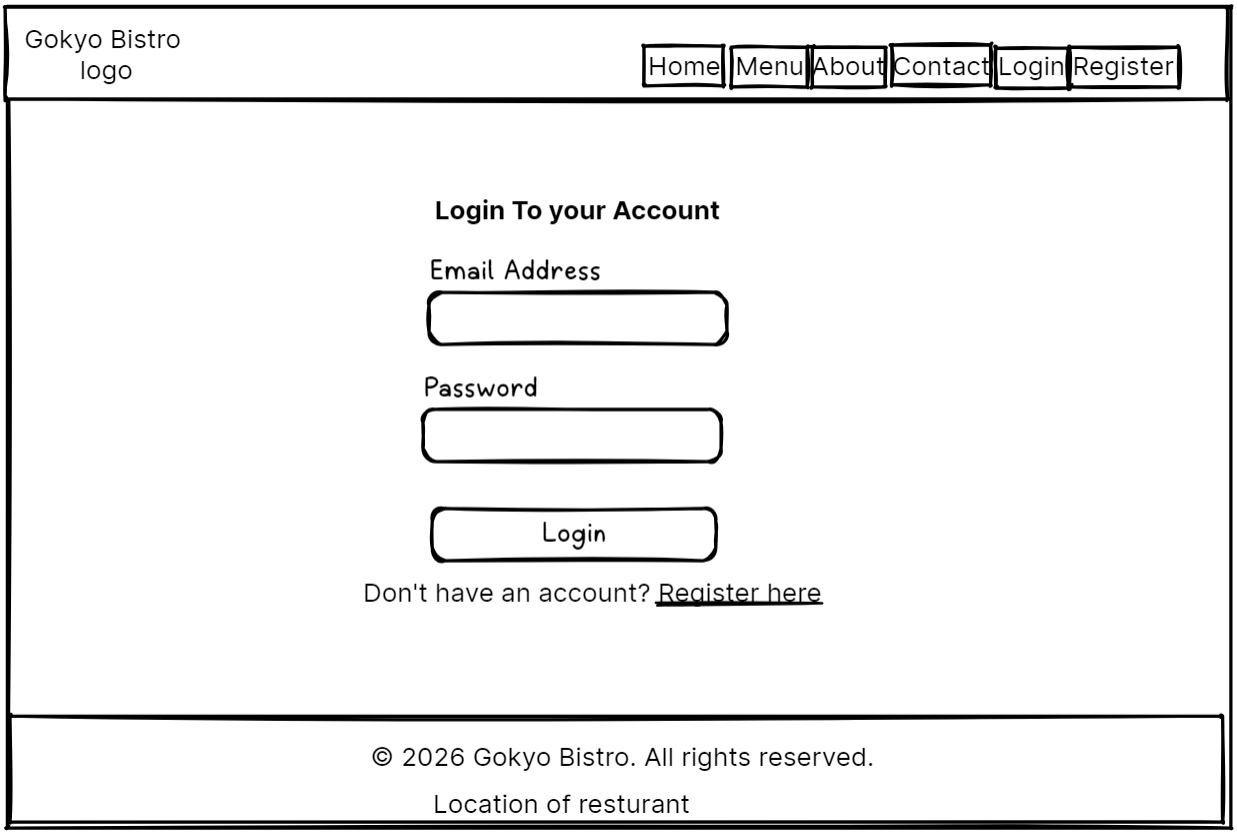


Figure 5:Login Page wireframe

This is the Login page of Gokyo Bistro website. It allows registered users to access their account by entering their email and password.

This wireframe helps me plan how the Login page will look before coding. It shows where email field, password field, and login button will be placed.

After successful login by the customers respective dashboard will be open, if the login by Admin then admin dashboard will be open or if login by member, then member dashboard will be open.

If the user is new, then there is also link provided below login button to open register page of this website to register new account.

3.6 Wireframe 6: Register Page Wireframe

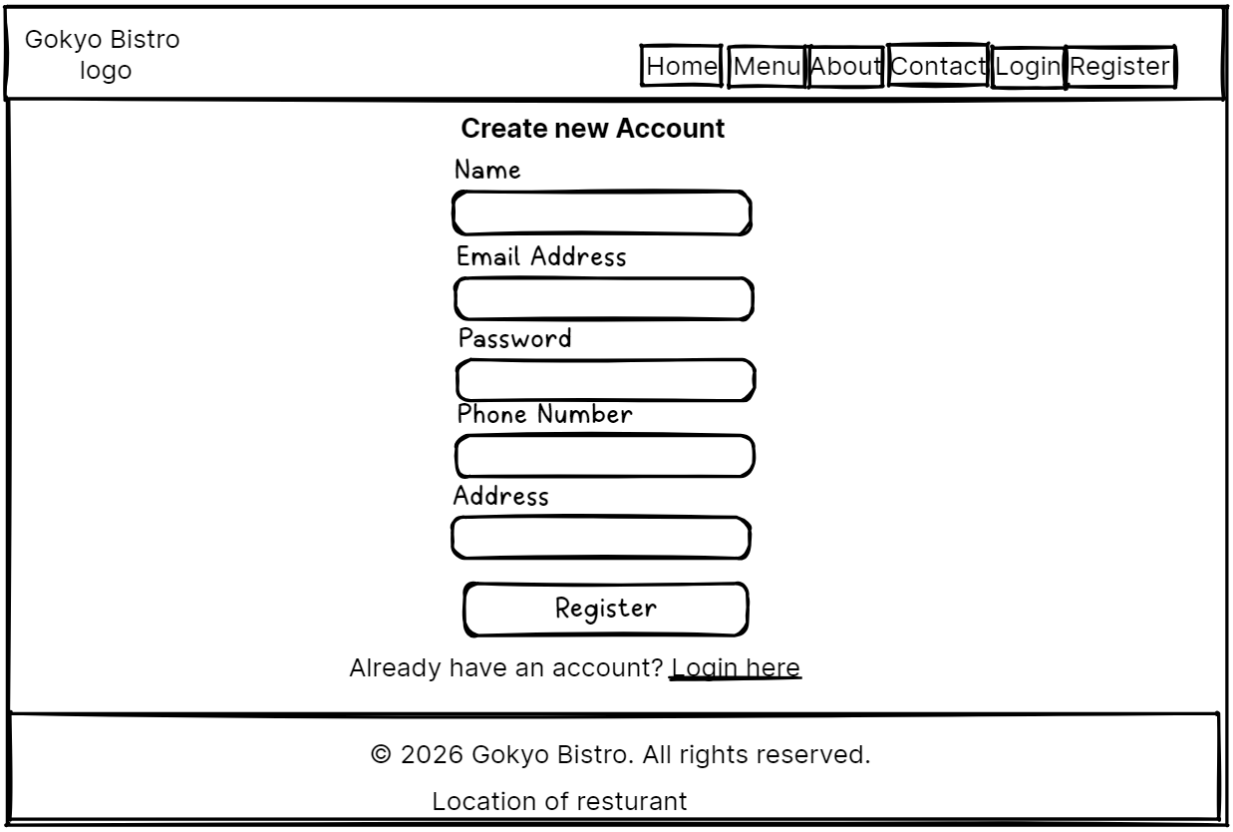


Figure 6:Register Page Wireframe

This page shows overall look when the website displays the register page where new users will be registering new account entering the details like name, email address, number, password, address then after clicking register button to submit registration form.

There is also provided link below register button to login directly without registration.

User fills the form then System checks if email already exists If email is new, account is created and Password is encrypted for security after that User is redirected to Login page where User can now login with their credentials.

3.7 Wireframe 7: Admin Dashboard Wireframe

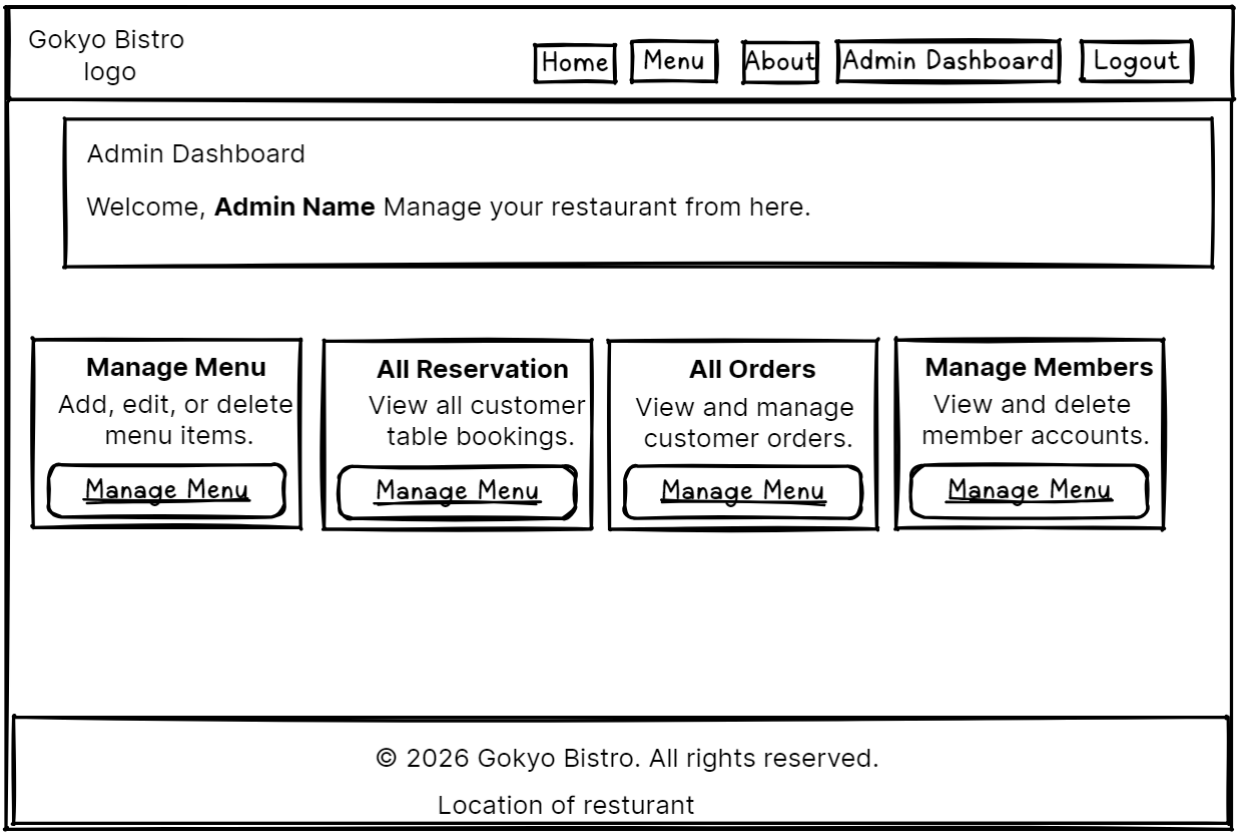


Figure 7:Admin dashboard Wireframe.

This wireframe helps to plan how the Admin Dashboard will look. It shows all the cards and features that admin needs to manage the restaurant.

This page gives overall view how admin dashboard will appear in website after successful login admin role based.

Herder section containing navigation to different pages where having navigation to log out from this admin dashboard. Also having welcome message to admin by their name for better feeling to admin members.

In the middle section having four box section to perform operations as Manage menu, All Reservation to view customer table bookings, All orders section to perform operation to view orders made by customers and Manage member section view and delete member account.

3.8 Wireframe 8: Member Dashboard Wireframe

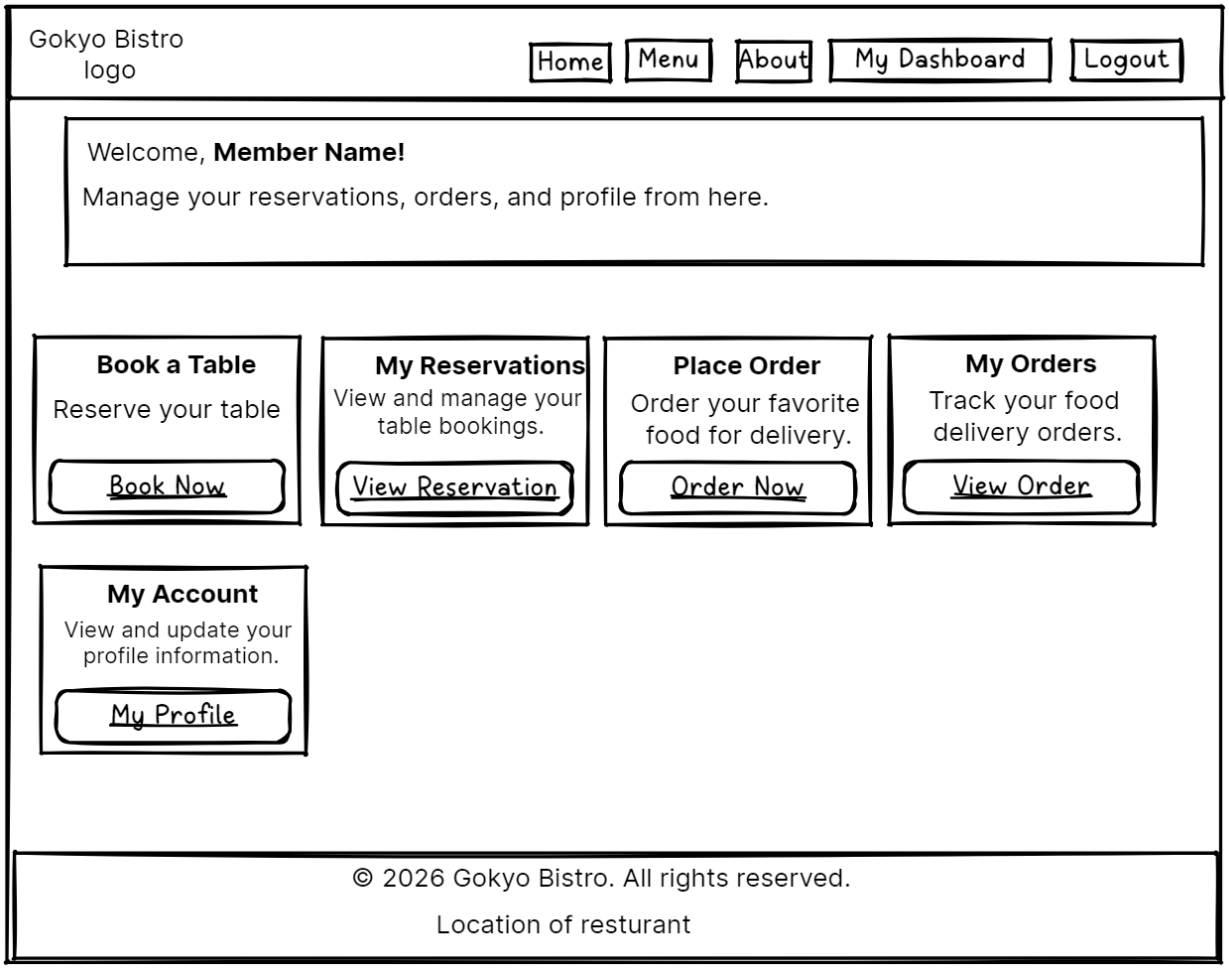


Figure 8:Member Dashboard Wireframe

This wire frame helps to plan the how the overall member dashboard (customer) will look like.

Header portion having logo with different pages navigation link with logout button.

In the middle having welcome message for member with their name for better felling for member by displaying their name. And below the welcome message having different section to perform operation by member.

Book table section helps to customer book their table by clicking on book now button provided. My Reservation section helps to customer to view their reservation. In Place order section, by clicking on order now button can order food for delivery. My order section where by clicking on view order button can view their order placed before. And last having My Account where member can mage their account.

3.9 Wireframe 9: Manage Meni Item Dashboard Wireframe

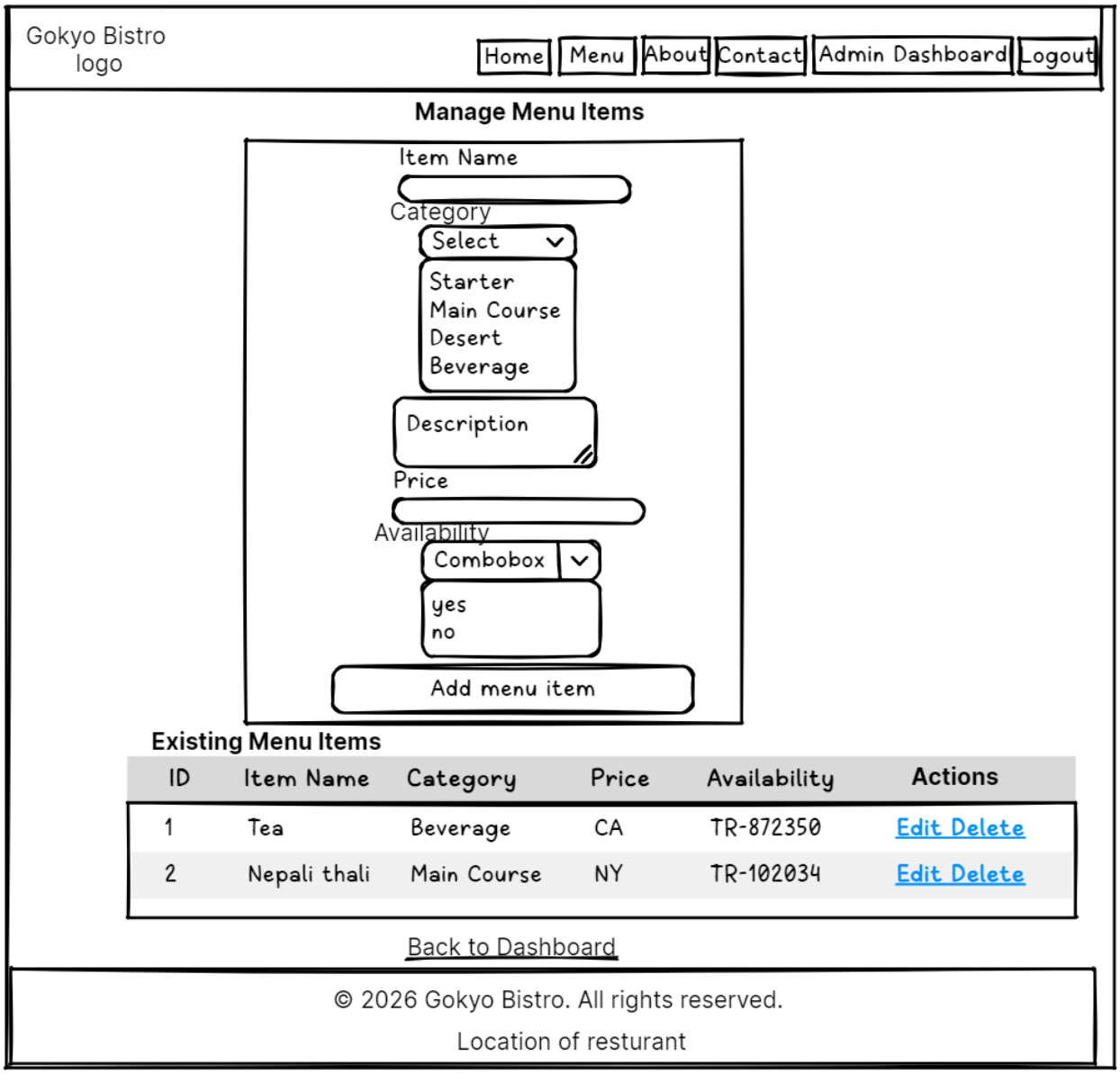


Figure 9:Manage menu item dashboard wireframe

This wire frame helps to plan the how the overall manage menu item dashboard will look like.

This is the Manage Menu page of Gokyo Bistro website. Only admin can see this page. Admin uses this page to add, edit, or delete food and drink items from the menu.

This page is needed because: -

- Add new items → When restaurant adds new food, admin can put it in menu.
- Update prices → If price changes, admin can edit it.
- Remove items → If food is not available, admin can delete it.
- Keep menu fresh → Restaurant can change menu anytime.

3.10 Wireframe 10: Manage Member Dashboard Wireframe

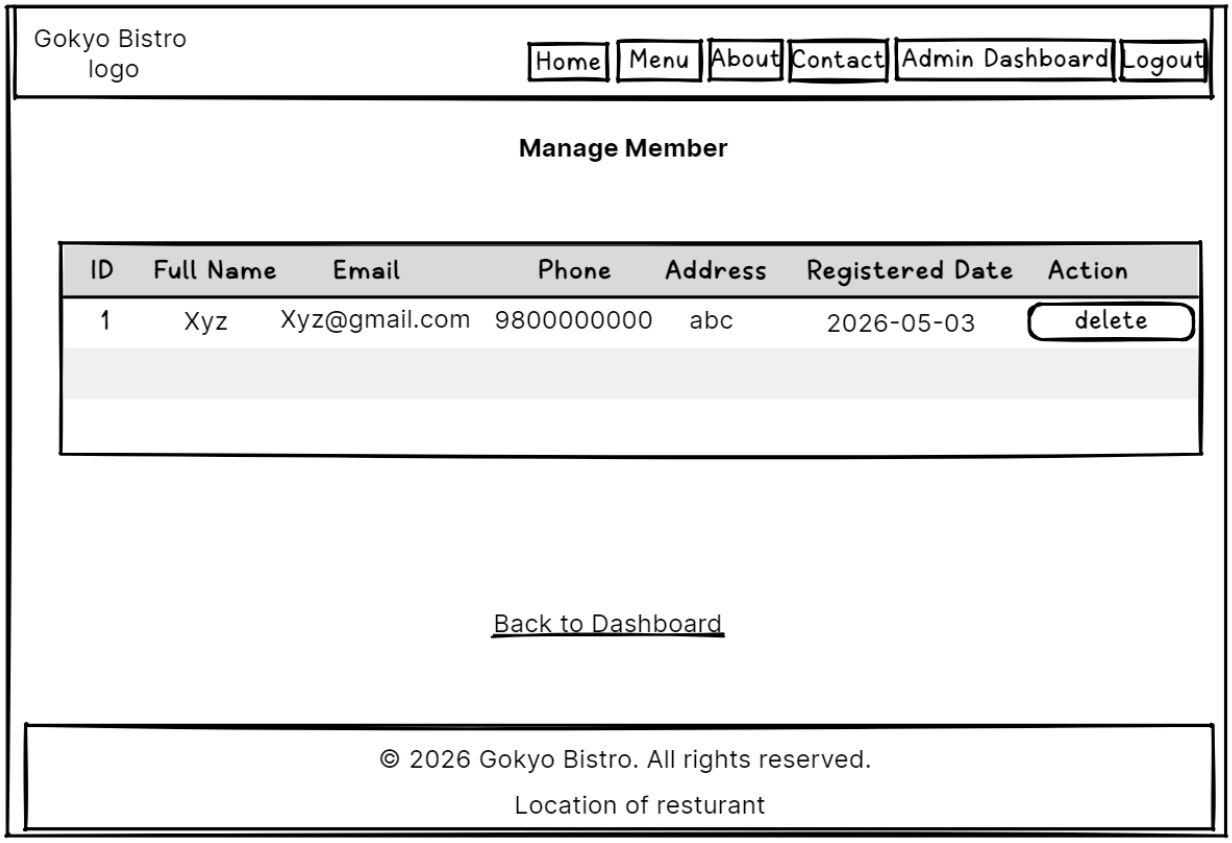


Figure 10:Manage member Dashboard wireframe

This is the Manage Members page of Gokyo Bistro website. Only admin can see this page. Admin uses this page to view all registered members and delete any member if needed. When a member is deleted then their past orders and reservations are also removed from the system.

The page shows a table with all members. The table has columns for member ID, full name, email, phone number, address, registration date, and a delete button. Admin can see all member details at one place. When admin clicks the delete button then a popup asks for confirmation before deleting the member. This page helps admin keep the member list clean and remove inactive users.

3.11 Wireframe 11: Book Table Dashboard Wireframe

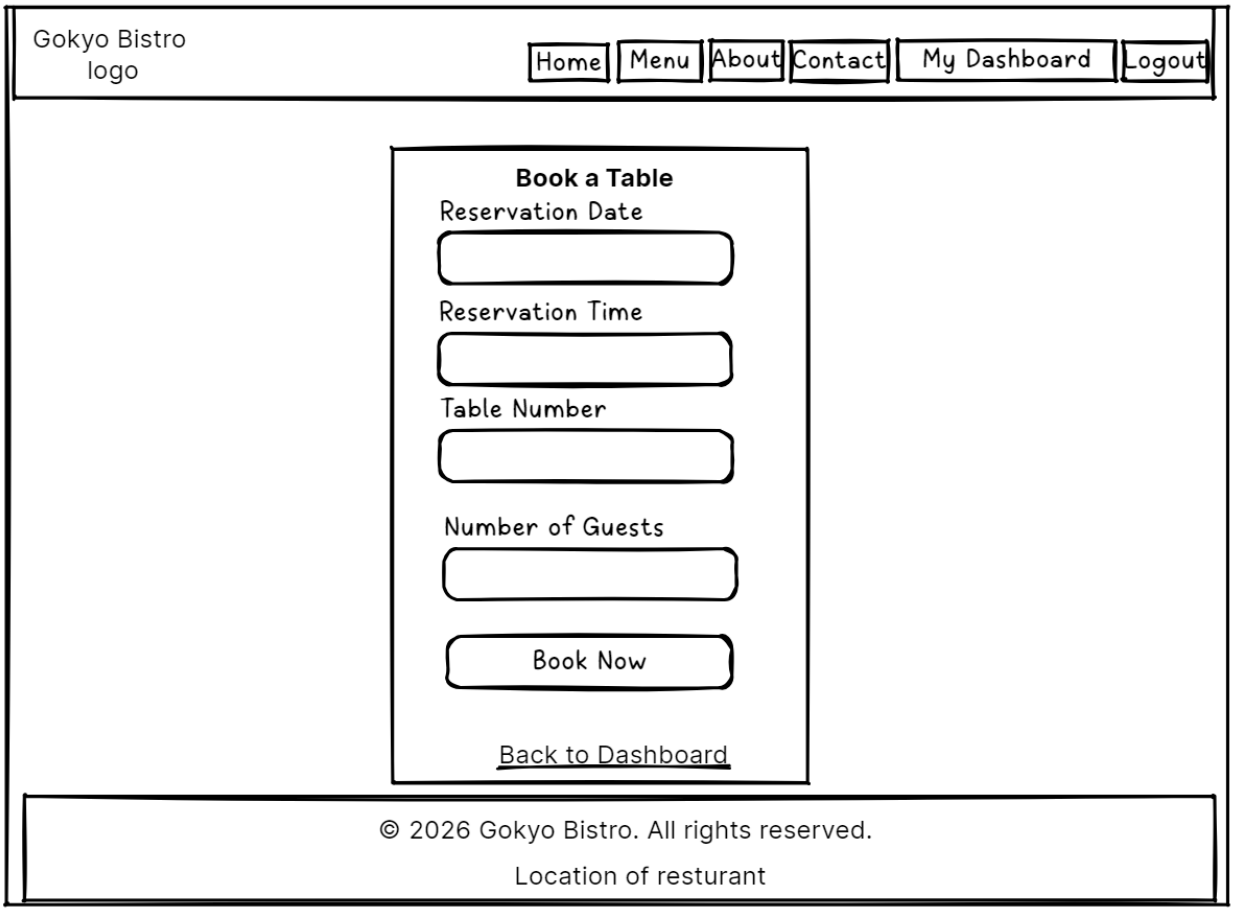


Figure 11:Book Table dashboard wireframe

This is the wireframe Book Table dashboard. Only logged in members can see this page. Visitors cannot book a table directly because they need to register first. Members use this page to reserve a table for dining at the restaurant.

By filling the details member can book their table. In Reservation date ask for date, Reservation time ask for desired time, Table number ask to select table number, number of guests ask for how many people will be come after booking in restaurants and finally after fulfilling above requirement can reserve the table by clicking Book now button.

3.12 Wireframe 12: Member account Wireframe

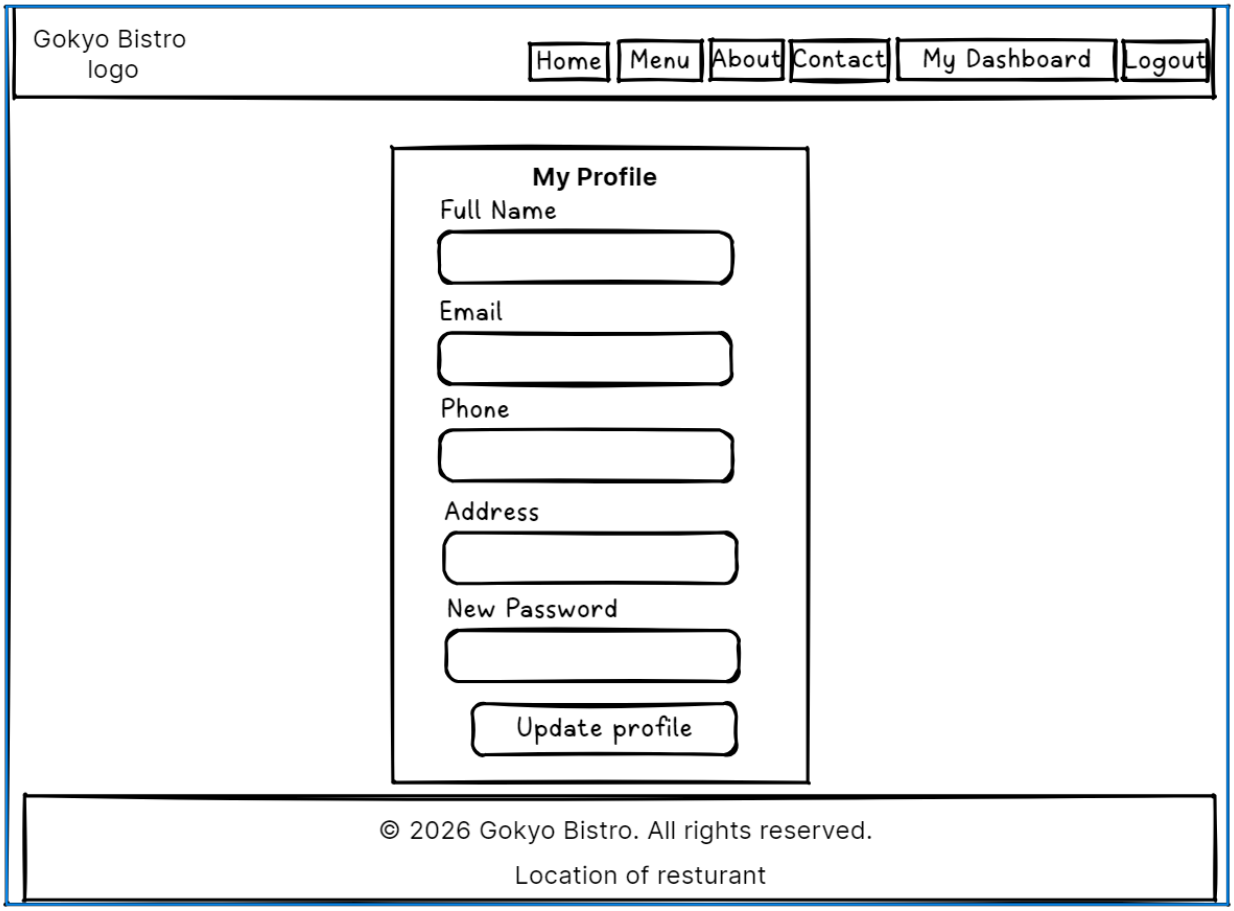


Figure 12:Member Account Wireframe

This wireframe shows how the dashboard will appear when member click their My account button from their dashboard. This page is only visible to member who logged in.

In this dashboard member can change their details while email cannot be changeable only full name, phone, address and password can be changeable. Member can leave the password field as it is if they don't want to change the password.

Then after clicking the update profile button, member can update their profile.

4. Java Classes and Methods Explanation

4.1 Class diagram of Model class

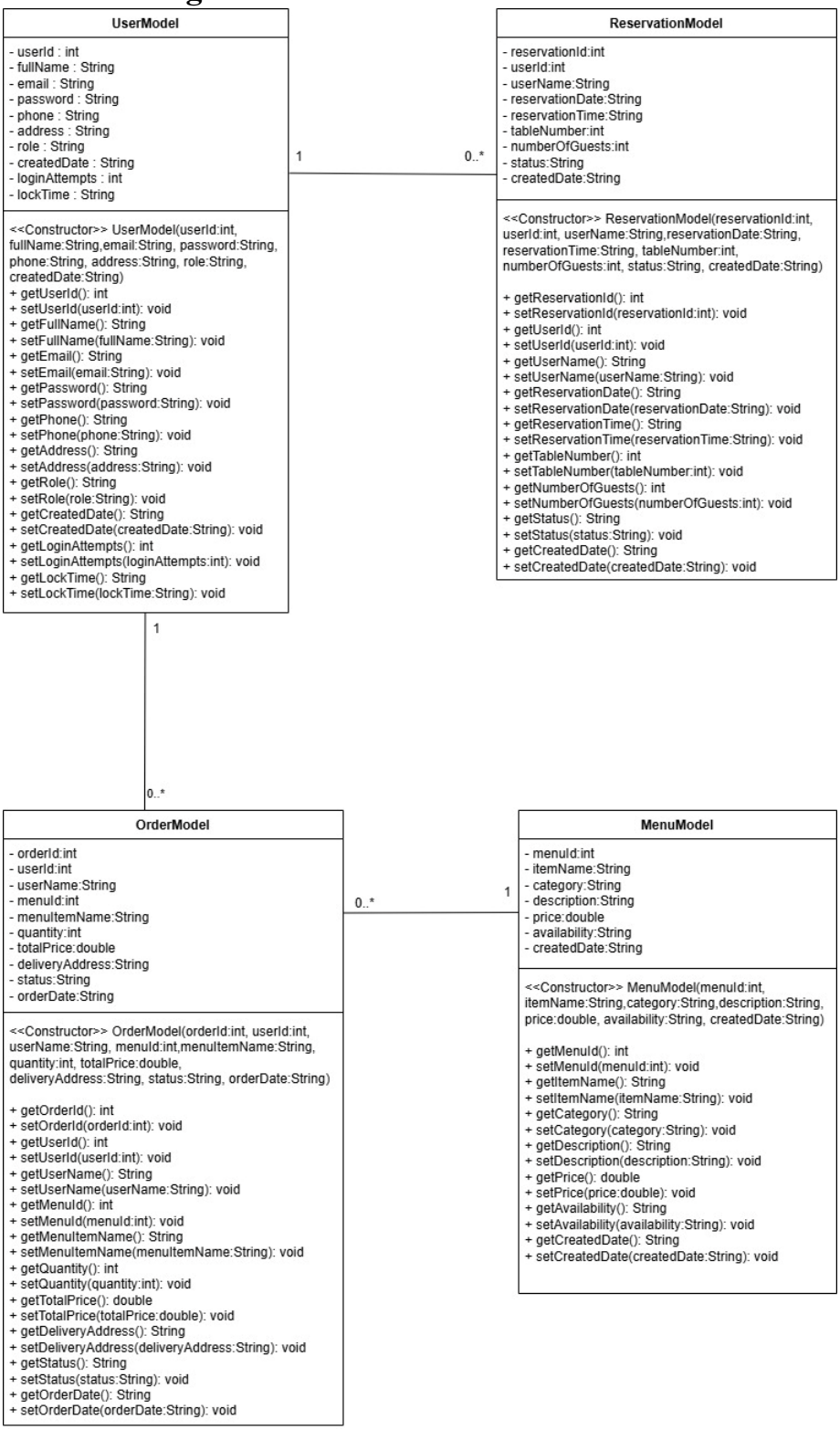


Figure 13:Class Diagram od Model classes

UserModel

→ This class used to store information who use this system as member by registering their account as well as for admin.

→ It keeps their name, email, password, phone, address and role. It also tracks failed login attempts and account lock time for security purposes.

MenuModel

→ This class used to store all food and drink items available in the restaurant. It keeps the item name, category (starter, main course, dessert, beverage), description, price and availability status.

ReservationModel

→ This class used to stores table booking records made by registered members.

→ It keeps the reservation date, time, table number, number of guests and status (confirmed, cancelled or completed).

OrderModel

→ This class helps to stores delivery orders placed by members.

→ It keeps which menu item was ordered, quantity, total price, delivery address and order status (pending, confirmed, delivered, cancelled).

4.2 Relationship between model classes

Relationship 1: UserModel → ReservationModel

→ Association type relationship.

→ One User(member) can have zero or many reservations.

→One Reservation belongs to one user.

Relationship 2: UserModel → OrderModel

→ Association type relationship.

→ One User(member) can have zero or many orders.

→ One order belongs to one user.

Relationship 3: MenuModel → OrderModel

→ Association type relationship.

→ One order belongs to at least one menu item.

→ One menu item can appear in zero or many orders.

4.3 Interactions Between Model Classes

→ Interactions mean how these classes work together during different operations in your Gokyo Bistro system.

Interaction 1: UserModel + ReservationModel (Booking a Table)

→ User logs into the system.

→ UserModel provides the userId of the logged user.

→ User selects date, time, table number and number of guest.

→ ReservationModel takes that user ID and booking details and will be saved in database.

→ UserModel tells who is booking. ReservationModel stores the booking.

Interaction 2: UserModel + OrderModel + MenuModel (Placing an Order)

→ User logs into the system.

→ UserModel provides the userId of the logged user.

→ User selects a food item from menu.

→ MenuModel provides the item price.

→ User enters quantity and delivery address.

→ OrderModel calculates total price (quantity * price).

→ OrderModel saves order with userId, menuId, quantity, total, address.

→ UserModel tells who is ordering, MenuModel tells what is ordered and price and OrderModel creates the order.

Interaction 3: UserModel + ReservationModel (Admin Viewing All Reservations)

→ Admin requests to see all reservations.

→ ReservationModel fetches all booking records.

→ Each reservation has a userId but no name.

→ UserModel provides the full name for that userId.

→ System combines reservation data and user name.

→ Admin sees complete reservations with customer names

→ ReservationModel gives booking data, UserModel gives customer names and together they show full information.

Interaction 4: UserModel + OrderModel + MenuModel (Admin Viewing All Orders)

→ Admin requests to see all orders.

→ OrderModel fetches all order records.

→ Each order has userId and menuId.

→ UserModel provides customer name for userId.

→ MenuModel provides item name for menuId.

→ System combines order data + customer name + item name.

→ Admin sees complete orders with customer and item names.

→ OrderModel gives order data, UserModel gives customer name and MenuModel gives item name. All three work together.

Interaction 5: UserModel + OrderModel (Member Viewing Own Orders)

→ Member logs into the system

→ UserModel provides the userId of the logged user.

→ OrderModel fetches only orders with that userId.

→ Member sees only their own orders.

→ UserModel provides user ID, OrderModel uses that ID to show only that users orders.

Interaction 6: ReservationModel + UserModel (Member Viewing Own Reservations)

→ Member logs into the system.

→ UserModel provides the userId of the logged user.

→ ReservationModel fetches only bookings with that userId.

→ Member sees only their own reservations.

→ UserModel provides user ID, ReservationModel uses that ID to show only that users bookings.

4.4 Method Explanation

→ This section explains the important methods used in the Gokyo Bistro system.

a) LoginController - doPost() method: -

→ This method runs when user clicks login button. It takes email and password from the form, checks them against database and logs user in.

→ It allows registered users to access their account and prevents unauthorized access.

Step wise how this method works

- get email and password from login form.
- check if account is locked due to 3 wrong attempts.
- If account is locked, then display error message showing account is locked.
- If not locked, then check email and password in database.
- If password and email match while checking in database, then respective dashboard will be opened based on role i.e. admin dashboard for admin and member dashboard for member.
- If password or email one of them is incorrect then error message display showing one of them is wrong and dashboard will not open.

b) RegisterController - doPost()

→ This method runs when user clicks register button. It takes user details from form and creates new account after that saves to database.

→ It allows new users to create an account and join the system.

Step wise how this method works

- Gets name, email, password, phone, address from form.
- Checks if email already exists in database.
- If email exists, display error message showing email already exist.
- If email is new, encrypt the password by calling restorerUser method from UserService.
- Saves user information to database.
- Redirects the recently registered user to login page.

c) MenuController - doGet()

→ This method runs when user opens menu page. It shows all food items and handles search.

→ It displays restaurant menu and allows customers to search for specific items.

Step wise how this method works

- Check if user typed something in search box.
- If search keyword exists, finds matching items.
- If there is no search keyword, shows all menu items.
- Send menu list to the menu page.

d) BookTableController - doPost()

→ This method runs when member books a table. It saves reservation details to database.

→ It allows members to reserve a table online.

Step wise how this method works

- get date, time, table number from form.
- get logged in user ID from session.
- check if table is already booked at that time.
- if available, saves reservation to database.
- show success message to user.

e) PlaceOrderController - doPost()

→ This method runs when member places a delivery order. It saves order details to database.

→ It allows members to order food for home delivery.

Step wise how this method works

- get menu item ID and quantity from form.
- get delivery address from form.
- get logged in user ID from session.
- calculate total price (price * quantity).
- save order to database.
- redirect to my orders page.

f) ManageMenuController - doGet()

→ This method runs when admin opens manage menu page. It shows all menu items in a table.

→ It allows admin to see all menu items at one place.

Step wise how this method works

- Check if admin is logged in.
- fetches all menu items from database.
- send menu list to the admin page.
- display items with edit and delete buttons.

g) ManageMenuController - doPost() for Add

→ This method runs when admin adds a new menu item.

→ It allows admin to add new food items to the menu.

Step wise how this method works

- get item name, category, description, price from form.
- set availability to Yes.
- save new item to database.
- show success message.

h) ManageMenuController - doPost() for Update

→ This method runs when admin updates an existing menu item.

→ It allows admin to change price or details of any menu item.

Step wise how this method works

- get menu ID and updated details from form.
- update the item in database.
- show success message.

i) AdminManageMembersController - doGet()

→ This method runs when admin opens manage members page. It shows all registered members.

→ It allows admin to see all registered members.

Step wise how this method works

- check if admin is logged in.
- fetches all users with role "member" from database.
- send member list to the admin page.
- show delete button for each member.

j) UserService - login()

→This method checks if email and password are correct in database.

→It verifies user credentials for login.

Step wise how this method works

- searche database for the email.
- if email found, gets stored password.
- compare entered password with stored encrypted password.
- if match, returns user information.
- if not match, returns null.

k) UserService - registerUser()

→This method saves new user to database.

→It stores new user accounts in database.

Step wise how this method works

- take user details from RegisterController.
- Encrypts password using PasswordUtil.
- insert user into database.
- return true if successful.

l) UserService - updatePasswordByEmail()

→ This method updates user password for forgot password feature.

→ It allows users to reset forgotten password.

Step wise how this method works

- take email and new password from ForgotPasswordController.
- Encrypts the new password.
- update password in database for that email.
- return true if successful.

m) MenuService – searchMenuItems()

→This method finds menu items matching search keyword.

→It allows customers to search for specific food items.

Step wise how this method works

- take keyword typed by user.
- create SQL query with LIKE condition.
- search item_name and category columns.
- return matching menu items.

n) ReservationService - bookTable()

→ This method saves table reservation to database.

→ It stores table booking records

Step wise how this method works

- take reservation details from BookTableController.
- insert reservation into database.
- return true if successful.

o) OrderService - placeOrder()

→ This method saves delivery order to database.

→ It stores food delivery orders.

Step wise how this method works

- take order details from PlaceOrderController.
- calculate total price.
- insert order into database.
- return true if successful.

p) PasswordUtil - encryptPassword()

→ This method converts plain password into encrypted format.

→ It keeps passwords secure and prevents storing plain text passwords.

Step wise how this method works

- take plain password from user.
- use SHA-256 algorithm to hash it.
- convert hash to hexadecimal string.
- return encrypted password.

q) AuthenticationFilter – doFilter()

→ This method checks if user is logged in before allowing access to pages.

→ It protects admin and member pages from unauthorized access.

Step wise how this method works

- run before every page request.
- check if requested page is public (home, login, register, menu).
- if public, allows access.
- if not public, checks if user is logged in.
- if not logged in, redirects to login page.
- if logged in, checks role for admin/member pages.
- it protects admin and member pages from unauthorized access.

5. Test Cases

The test cases cover functional testing, validation testing, exception handling testing, and UI testing. Followings below tables showings test cases performed on this project dynamic web developed system project.

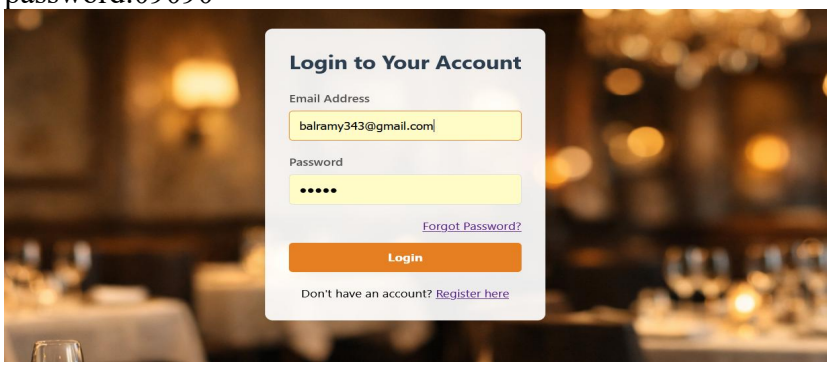
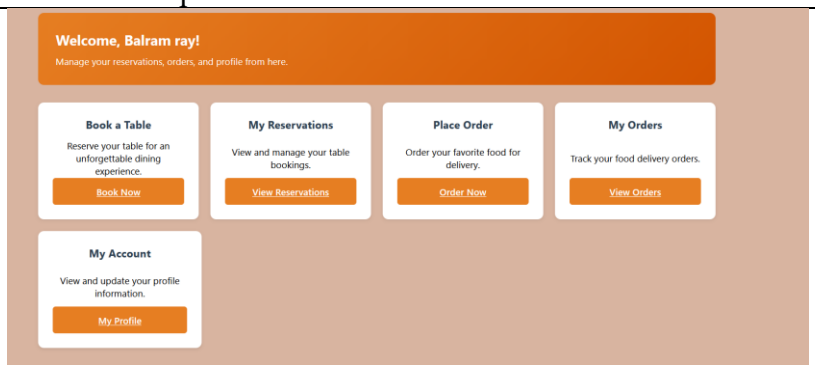
5.1 Functional Testing

Functional testing checks whether the function is working as expected properly or not.

Test Case 1: Valid User Login

This test is done to check whether after putting correct email address and password to login account page to access respective dashboard. For registered member, their respective dashboard should open and for admin their respective admin dashboard should open.

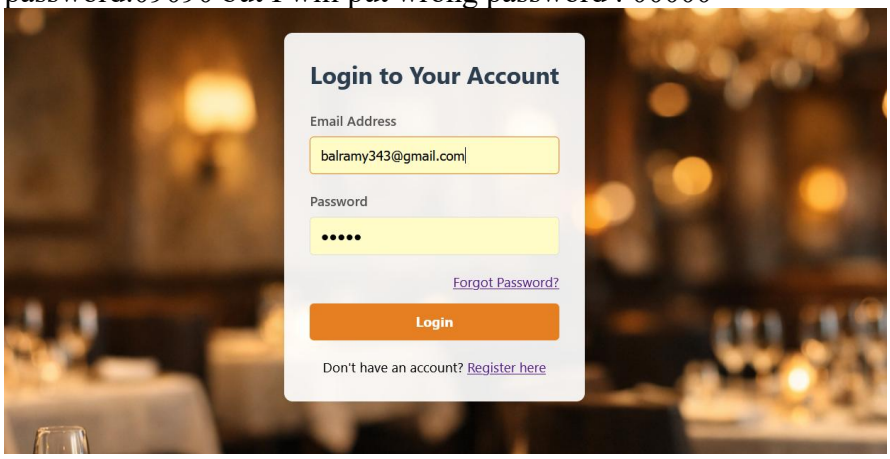
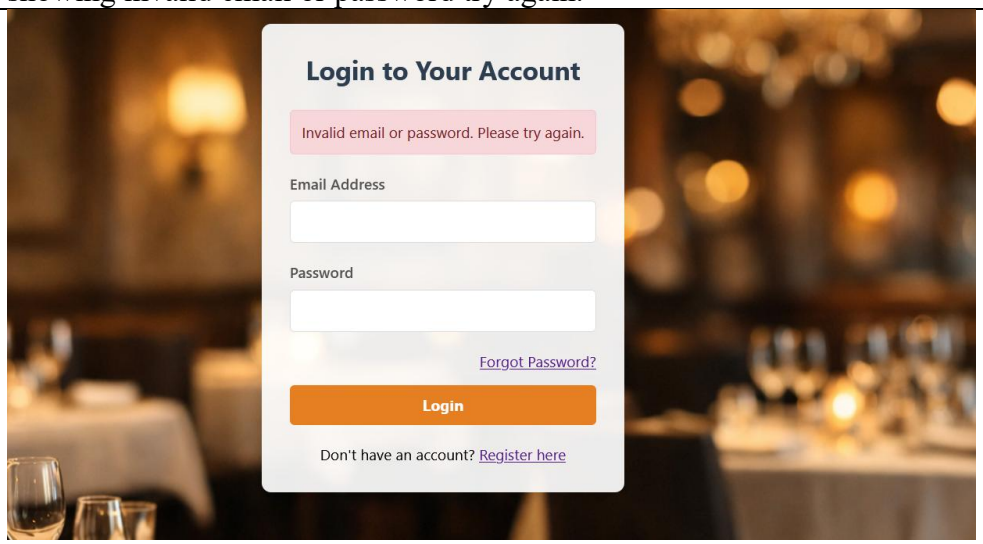
Table 1:valid user login (test 1)

Test ID	1
Test Description	Verify that a registered user can login successfully with correct email and password and directed toward the member dashboard successfully.
Input Data	I have already registered a user having email: balramy343@gmail.com with password:09090  This picture shows that the how input data is kept in login page to access ther respective dashboard.
Expected Output	if registered member login then should open member dashboard , if admin login then should open admin dashboard.
Actual Output	 This picture shows through correct password and email provided by registered member, member dashboard is opened.

Test Case 2: User Login with Wrong Password

This test is performed to check if the user login with wrong email or password they should not directed to their respective dashboard, must show error message that "wrong email or password, try again". This test shows how security purpose concept is integrated in this system to access the system from only valid users.

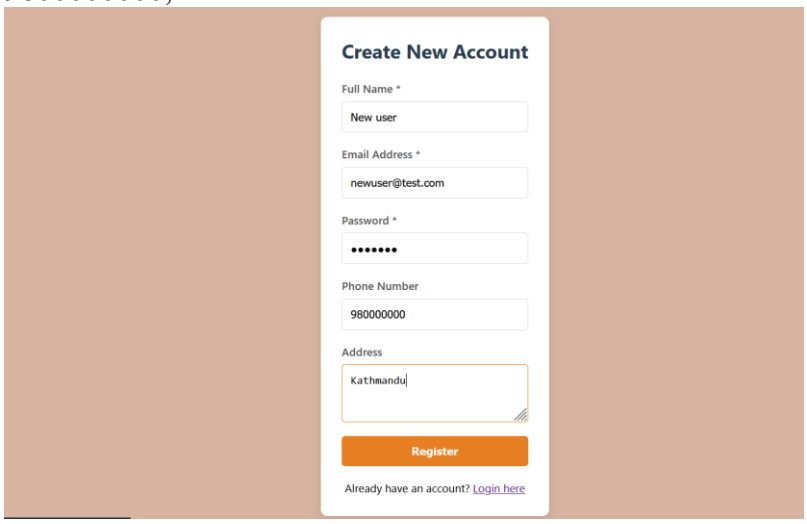
Table 2:user login with wrong password(test2)

Test ID	2
Test Description	Verify that a registered user cannot login with wrong email or password and directed toward the member dashboard successfully.
Input Data	I have already registered a user having email: balramy343@gmail.com with password:09090 but I will put wrong password : 00000  This picture shows that the how input data is kept in login page to access ther respective dashboard.
Expected Output	if registered member login with wrong password message should display showing invalid email or password try again.
Actual Output	 This picture shows through wrong password or email provided by registered member to login then invalid message displayed to user.

Test Case 3: New User Registration

This system only gives access to registered user to make order and book table facility, if visitor want to get this benefit to order food for home delivery or to make table reservation they should first open new account through register page and only after login they can get dashboard to order food and to book table.

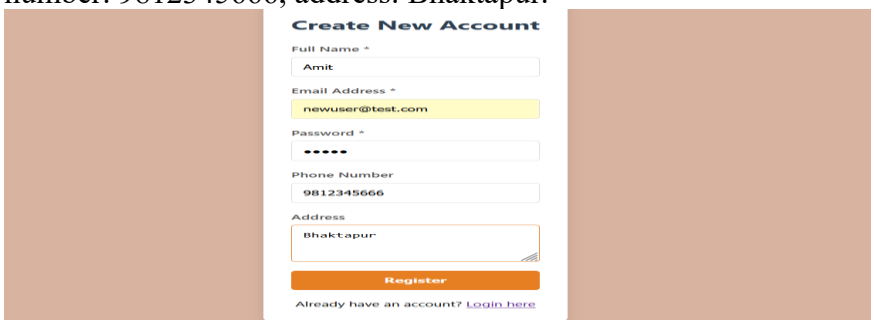
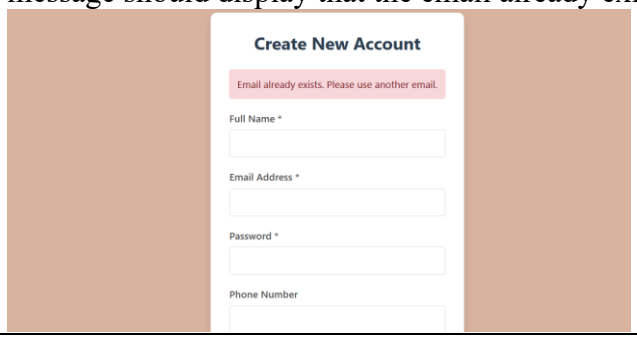
Table 3:new user registration(test3)

Test ID	3
Test Description	Verify that a visitor wants to open new account by fulfilling by putting valid details to create new account through by clicking on register by providing full name, email address, password, phone number , address and by clicking on register button to create new account.
Input Data	Name: New User, Email: newuser@test.com, Password: pass123, Phone: 9800000000, Address: Kathmandu.  This picture shows how input data is kept in the register page to create new account.
Expected Output	After fulfilling the details in register page new account will create successfully and user will be redirect to login page.
Actual Output	Account saved in database, redirected to login page.

Test Case 4: Registration with Existing Email

If new user wants to register their account to access the benefit provided by member to book table and to order food for home deliver, they first register their account though register page dashboard, where they should provide valid details and one thing they should provide unique email if the provided email is matching the existing email then dashboard should display message regarding the enter unique email because provided email already exist. Below table showing how I tested these activities to ensure that the system does not allow duplicate email to register new account.

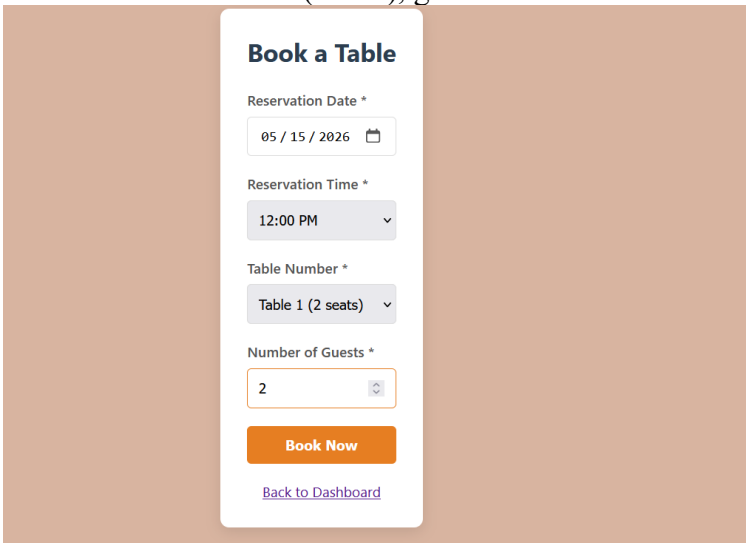
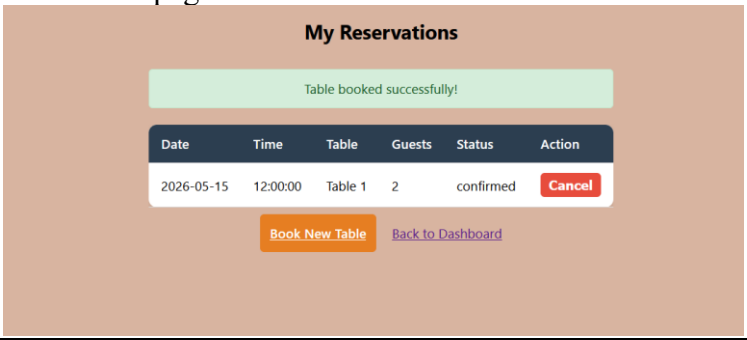
Table 4: Registration with existing email(test4)

Test ID	4
Test Description	This test shows that the system will not allow to registered new account with existing email, this test make sure system ensure security concept used.
Input Data	I am providing the email already exist with other input data by changing here email already exist. Full name: Amit, email address: newuser@test.com, password: 12345, Phone number: 9812345666, address: Bhaktapur.  This picture shows how input data is kept in the register page to create new account. Here same existing email is kept and try to register.
Expected Output	After fulfilling the details in register page where by changing other details like full name, address, password, address but kept email already exist then, error message should display that the email already exist. 
Actual Output	Error message displayed correctly.

Test Case 5: Member Books Available Table

This system allows the registered member to book the available table for their enjoyment with their favorite ones to make beautiful in their desired date. This test checks whether a member can successfully book a table that is free at their chosen date and time.

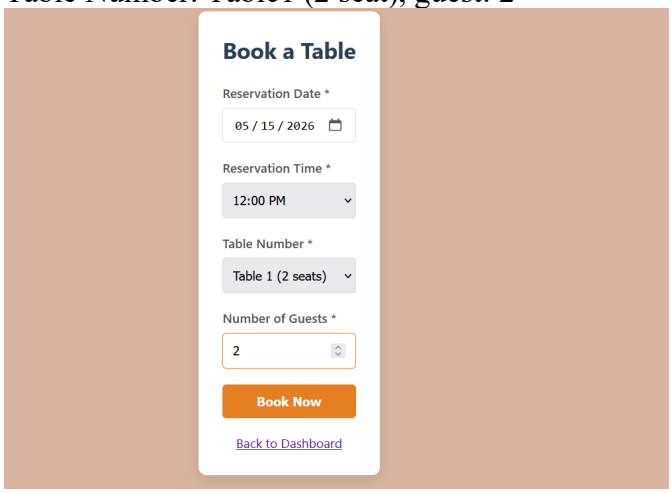
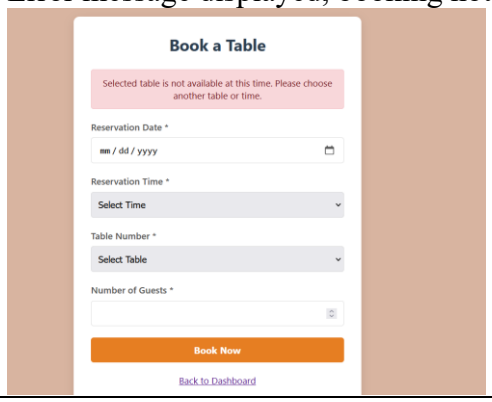
Table 5: Member book available table (test5)

Test ID	5
Test Description	This test shows that the system allows to book table to registered member to available date, time and for available table.
Input Data	Providing details in the book table page to book table for newuser@test.com Reservation date: 05-15-2026, Reservation Time :12 PM, Table Number: Table1 (2 seat), guest: 2  This picture shows how input data is kept in Book table page to create new booking by registered user.
Expected Output	After fulfilling the details in book table page to book table, Booking confirmation message and reservation saved to database. And will display details in my reservation page.
Actual Output	Booking saved successfully, reservation ID generated and displayed in my reservation page. 

Test Case 6: Member Books Already Booked Table

When a member tries to book table already booked by other member for same date, same time and for same table then proper messgase should display showing that the desired table already booked by another member.

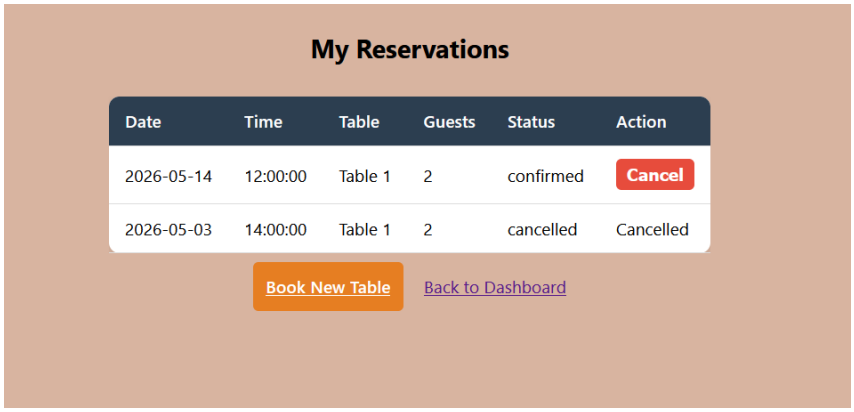
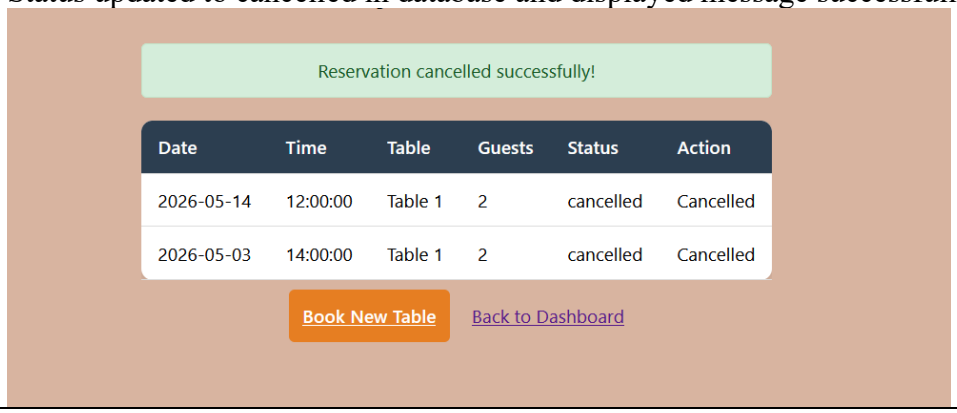
Table 6:Member Books already booked table (test6)

Test ID	6
Test Description	This test shows that the system does not allow to book table to member to available date, time and for table which already booked for specified time, date.
Input Data	Providing details in the book table page to book table for balramy343@gmail.com Reservation date: 05-15-2026, Reservation Time :12 PM, Table Number: Table1 (2 seat), guest: 2  This picture shows how input data is kept in Book table page to book already booked table for specified date and time.
Expected Output	After fulfilling the details in book table page to book already booked table, Error message "This table is already booked for the selected date and time." Should display.
Actual Output	Error message displayed, booking not saved. 

Test Case 7: Member Cancels Reservation

Members who booked table previously can cancel their reservation by clicking cancel button for desired booked table from my reservation page from this system.

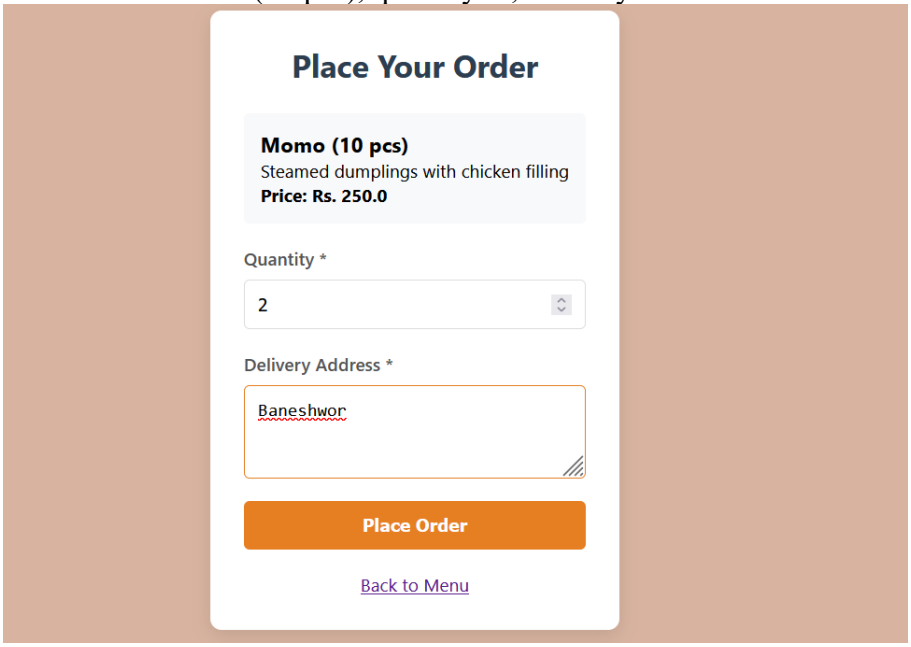
Table 7:Member cancel their reservation (test7)

Test ID	7
Test Description	Verify that a member can cancel their existing reservation.
Input Data	Member will open my reservation dashboard and click cancel button to cancel their reservation. 
Expected Output	Reservation status changed to "cancelled" and will be updated in database and to admin dashboard too and should display message that reservation cancelled successfully.
Actual Output	Status updated to cancelled in database and displayed message successfully. 

Test Case 8: Member Places Delivery Order

In this system members can place orders for home delivery to enjoy their food without going to restaurants, which saves their time. This system will be best option for people who want to enjoy their food without going to restaurant.

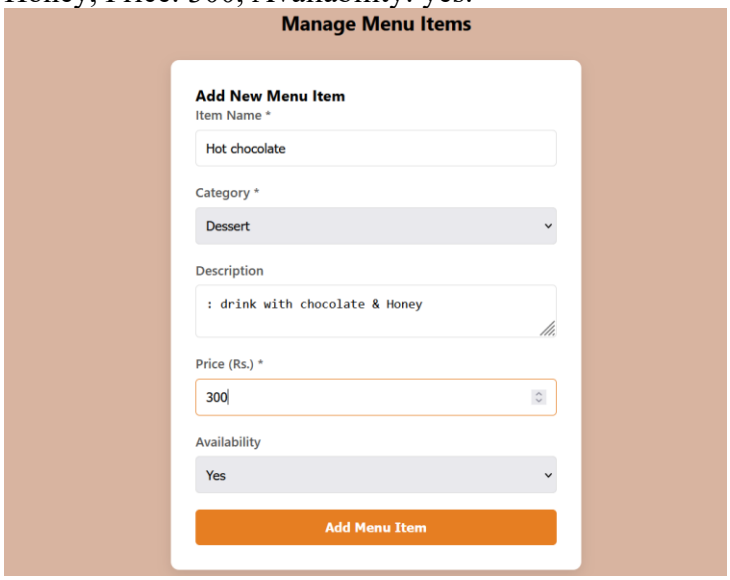
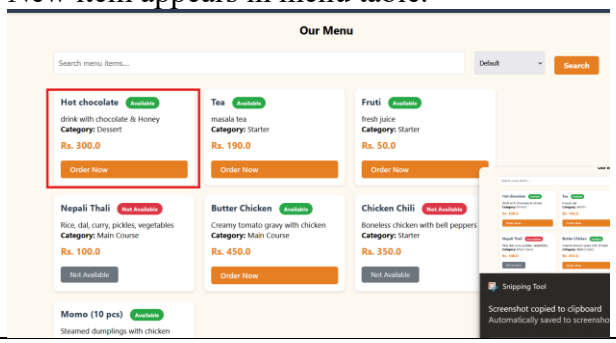
Table 8:member places delivery order (test8)

Test ID	8
Test Description	Verify that a member can place food order for home delivery.
Input Data	Member will open my reservation dashboard and click to my orders then selecting items they want to order by order now button for desired items, place your order screen will appear with input field quantity and delivery order. Order item: Momo (10 pcs), quantity: 2, delivery address: Baneshwor. 
Expected Output	Order saved successfully, order ID generated, redirect to my orders page.
Actual Output	Order appears in my orders list of members who ordered the food for delivery. 

Test Case 9: Admin Add New Menu Item

Admin Add New Menu Item is a feature that allows the restaurant administrator to add new food or drink items to the menu list. When the restaurant wants to introduce a new dish then admin can simply fill a form with item name, category, price, description and availability. Once submitted, the new item is saved to the database and immediately appears on the customer menu page. This feature helps the restaurant keep its menu updated with fresh items without any technical help.

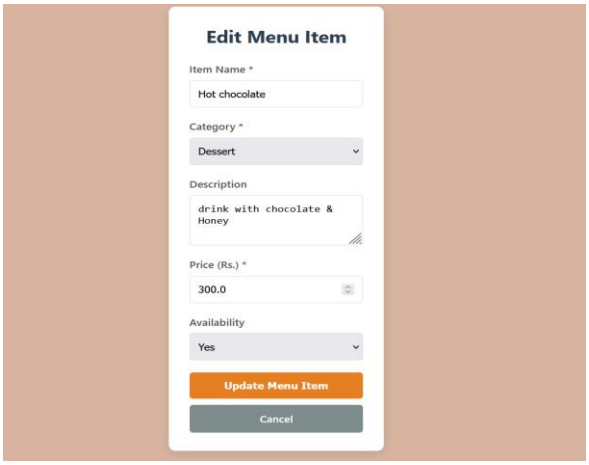
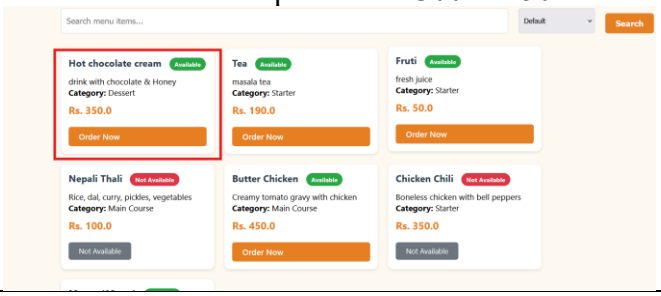
Table 9:Admin add new menu item (test9)

Test ID	9
Test Description	Verify that admin can add a new food item to the menu. After admin clicking button in manage menu then the dashboard will open with input filed to take input to add new items to the menu.
Input Data	After opening the manage menu item dashboard admin fills the input filed. Suppose admin want to add new item “hot chocolate” Item Name: Hot chocolate, category: desert, description: drink with chocolate & Honey, Price: 300, Availability: yes.  This figure shows how to manage menu items dashboard will open and how to add new items by fulfilling the input fields.
Expected Output	After fulfilling all the input fields and clicking Add menu Item, new menu item added to database and appears in menu table.
Actual Output	New item appears in menu table. 

Test Case 10: Admin Edits Menu Item

Admin Edits Menu Item is a feature that allows the restaurant administrator to update existing food or drink items on the menu. When the price of a dish changes or when the restaurant wants to modify item details like name, category, description or availability then admin can simply click the edit button next to any menu item, make the necessary changes and save them. Once updated, the changes are immediately reflected on the customer menu page. This feature helps the admin keep menu information accurate and up to date without any difficulty.

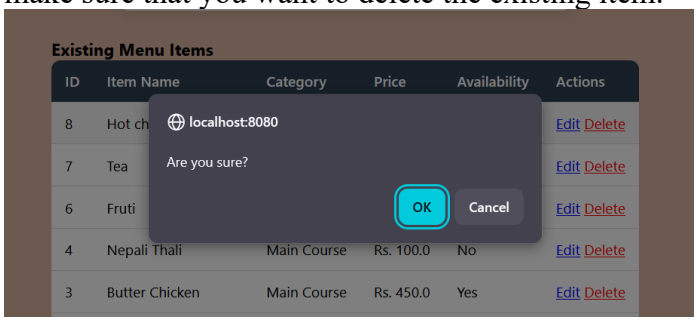
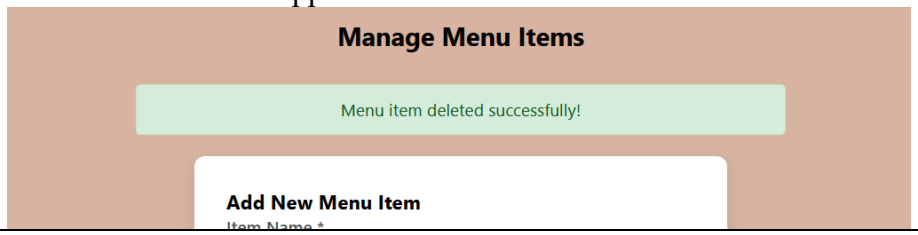
Table 10:Admin Edits Menu Item (test 10)

Test ID	10
Test Description	Verify that admin can edit existing menu item details. After admin clicking button in manage menu then the dashboard will open with input filed to take input to add new items to the menu and below this add menu item form existing menu item containing menu items table.
Input Data	After clicking on edit button adjacent to delete button, manage menu item form dashboard will open where admin can edit the detail. After clicking Hot chocolate, the below figure showing how dashboard will open where I want to change price from 300 to 250 and changing name from hot chocolate to Hot chocolate cream. 
Expected Output	After changing the details and after clicking update menu item, ,menu item will update in database as well as in Our menu dashboard.
Actual Output	Here picture showing that items is updated with name from hot chocolate to hot chocolate cream and price from 300 to 250. 

Test Case 11: Admin Deletes Menu Item

Admin Deletes Menu Item is a feature that allows the restaurant administrator to remove food or drink items from the menu. When a dish is no longer available or the restaurant decides to stop serving a particular item then admin can simply click the delete button next to that menu item. Once deleted the item is removed from the database and no longer appears on the customer menu page. This feature helps the admin keep the menu clean and show only those items that are currently available for customers to order.

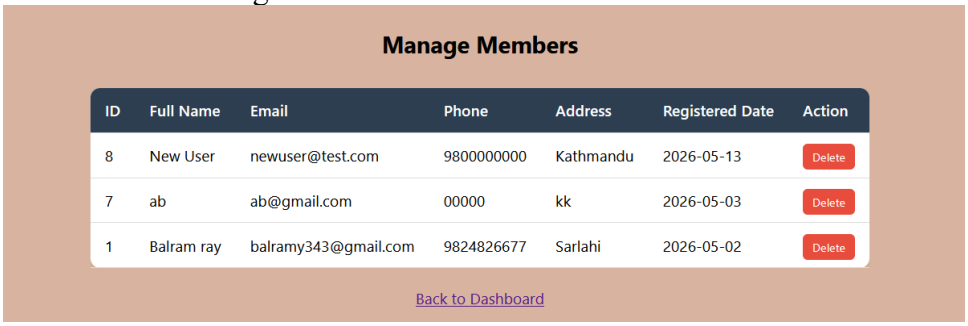
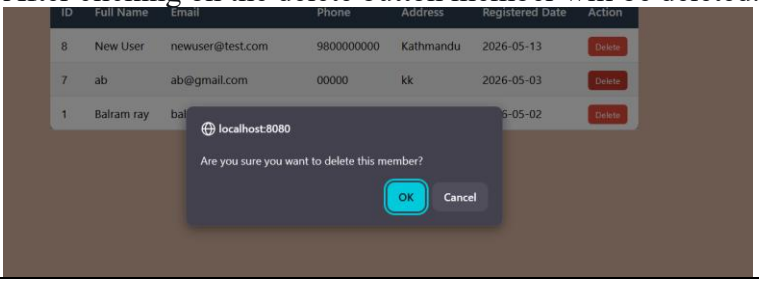
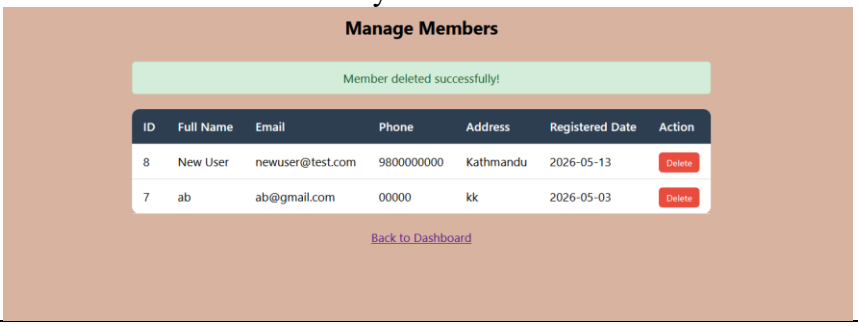
Table 11:Admin Deletes Menu item (test 11)

Test ID	11
Test Description	Verify that admin can delete existing menu item from menu. After admin clicking button in manage menu then the dashboard will open with input filed to take input to add new items to the menu and below this add menu item form existing menu item containing menu items table.
Input Data	After clicking on delete button adjacent to edit button, a pop box will open and make sure that you want to delete the existing item. 
Expected Output	After clicking delete button a pop up box asking to make sure to delete the item, after clicking to ok, the items will delete from database as well as disappear from the our menu dashboard also.
Actual Output	Items deleted and disappear from the menu items 

Test Case 12: Admin Deletes Member

Admin Deletes Member is a feature that allows the restaurant administrator to remove registered members from the system. When a member requests account deletion or if there is an inactive or fake account then admin can simply click the delete button next to that member name. When a member is deleted. This helps keep the member list clean and manage only active users in the system.

Table 12:Admin Deletes member (test13)

Test ID	12
Test Description	Verify that admin can delete a registered member from the system. After admin clicking button in manage member then the manage member dashboard will open with members details where admin can see the registered members details in table format.
Input Data	After clicking on delete button, a pop box will open and make sure that you want to delete the existing item. 
Expected Output	After clicking delete button a pop-up box asking to make sure to delete this member. That will delete member with all orders and reservations. After clicking on the delete button member will be deleted. 
Actual Output	Members deleted successfully 

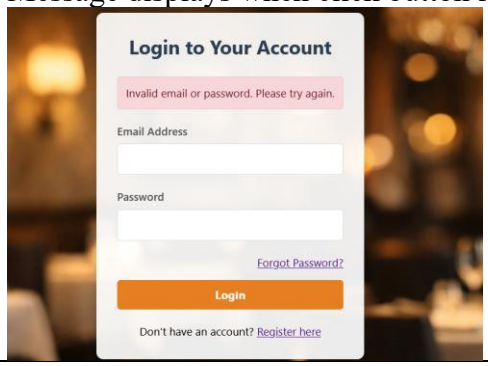
5.2 Validation Testing

Validation testing checks whether the system rejects invalid inputs.

Test Case 13: Login with Empty Fields

This test checks whether the login page shows an error message when a user tries to login without entering email or password. When the user leaves both fields blank and clicks the login button, the system displays a popup alert saying, "Please fill all required fields". This prevents the user from submitting an empty form and helps them understand what information is missing**.**

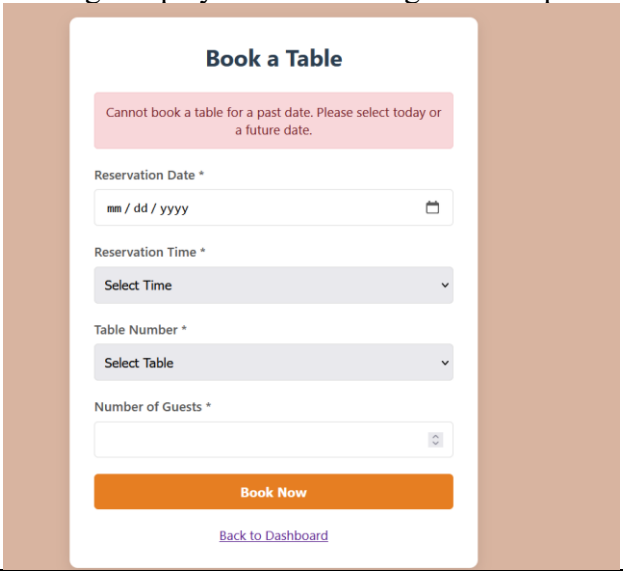
Table 13: Login with empty field (test 13)

Test ID	13
Test Description	Verify that login does not work when email or password fields are empty. When user try to click login button in login dashboard without filling the email fields and password filed. Login button don't works.
Input Data	No email entered, no password entered
Expected Output	When user click login button without filling the email and password field then message should display the please fill all the required filed.
Actual Output	Message displays when click button login with empty form. 

Test Case 14: Book Table with Past Date

Book Table with Past Date is a validation feature that prevents members from booking a table on a date that has already passed. When a member selects a past date (yesterday, last week, or any previous date) and clicks the book button, the system checks the date and shows an error message saying "Cannot book a table for a past date. Please select today or a future date." This ensures that customers can only make reservations for today or upcoming days, which is logical for restaurant operations.

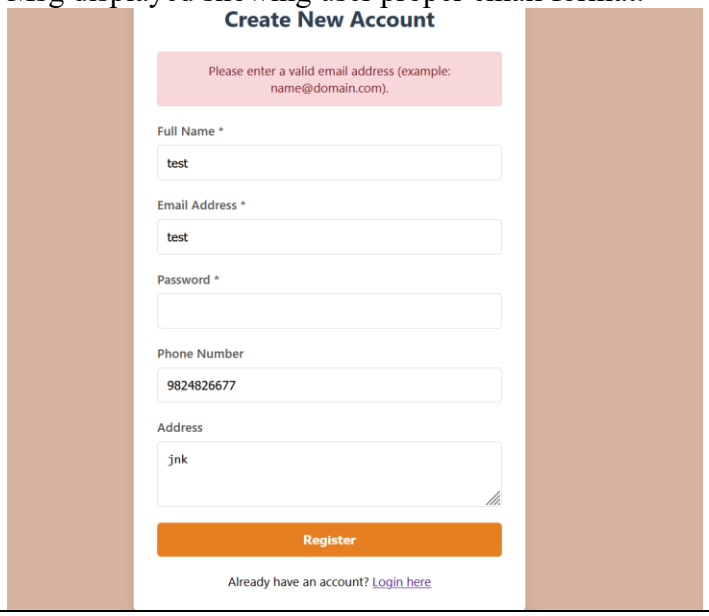
Table 14: Book Table with past date (test 14)

Test ID	14
Test Description	Verify that members from booking a table on a date that has already passed. When member try to book table by selecting past date then, system will allow to book table for past date.
Input Data	While Booking table take paste date.
Expected Output	When try to book table for past date, error message should display showing “Cannot book a table for a past date. Please select today or a future date.”
Actual Output	Message displays while booking table for past date. 

Test Case 15: Email Format Validation

This test falls under Validation Testing. The purpose is to check whether the registration system validates email format correctly. When a user enters an email without @ symbol like "testuser" and clicks register, the system detects the invalid format, shows an error message and prevents registration. This ensures only properly formatted emails are accepted, which is essential for future communication like password reset and order confirmation.

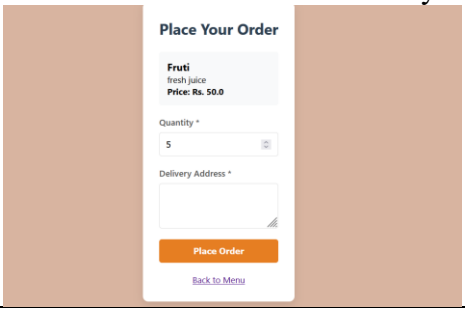
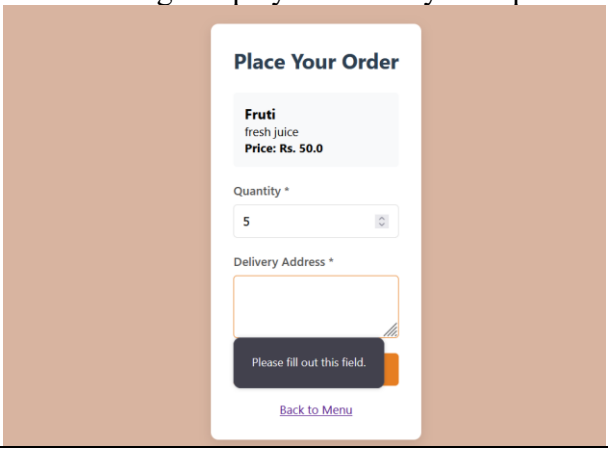
Table 15:Email format validation (test15)

Test ID	15
Test Description	Verify that the system accepts only properly formatted email addresses during registration and rejects invalid ones. When visitor try to make new account trough register page from this system then system should take only valid format email.
Input Data	Invalid email: "testuser" (without @ symbol)
Expected Output	When try to register new account through without proper valis email then should display error message. Error message "Please enter a valid email address (example: name@domain.com)"
Actual Output	Message displays while registering new account through invalid email format. Msg displayed showing user proper email format. 

Test Case 16: Place order with empty Address.

This test falls under Validation Testing. When a member selects a food item and quantity but leaves the delivery address field empty, the system detects the missing address, shows an error message and prevents the order from being placed. This ensures every delivery order has a valid address for delivery personnel to reach the customer.

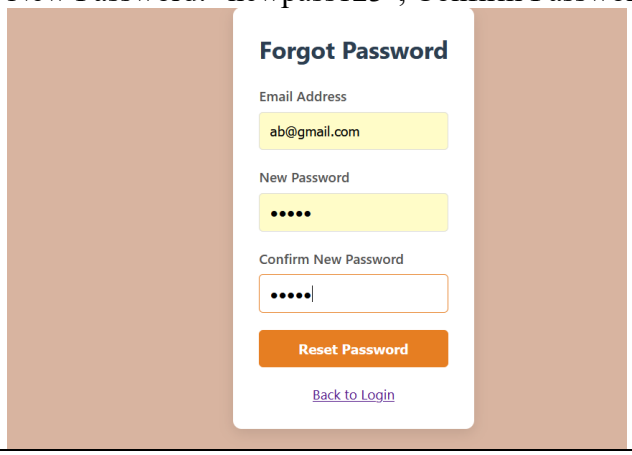
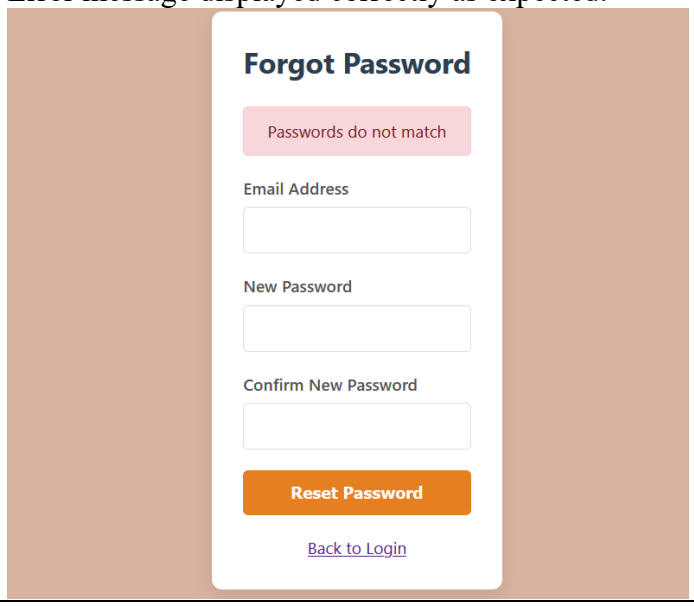
Table 16: Place order with empty field (test 16)

Test ID	16
Test Description	Verify that delivery order cannot be placed without delivery address. When registered members try to order food delivery through place order dashboard then they should provide proper delivery address because without proper address deliver can't happen.
Input Data	Place order Fruti without delivery address i.e. empty delivery address 
Expected Output	When registered member try to order Fruti without deliver address provide i.e. empty filed then error message should display.
Actual Output	Error message displayed correctly as expected. 

Test Case 17: Forgot Password - Password Mismatch Testing

This test falls under Validation Testing. When a user tries to reset their password, they must enter the same password in both "New Password" and "Confirm New Password" fields. If the passwords do not match, the system detects the mismatch, shows an error message "Passwords do not match" and prevents the password from being updated. This ensures the user does not accidentally set an incorrect password.

Table 17: Forgot Password- Password Mismatch Testing (test 17)

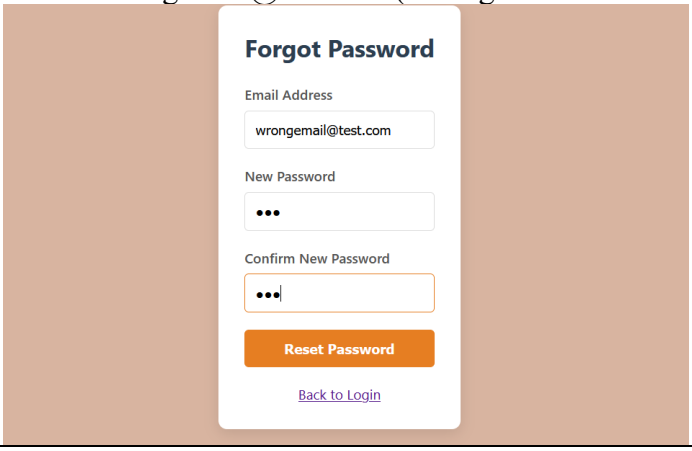
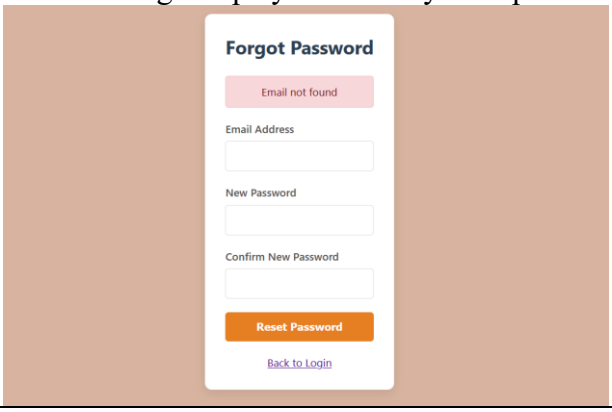
Test ID	17
Test Description	Verify that the system shows error message when new password and confirm password do not match.
Input Data	Email: ab@email.com, New Password: "newpass123", Confirm Password: "different456" 
Expected Output	Error message "Passwords do not match".
Actual Output	Error message displayed correctly as expected. 

5.3 Exception Handling Testing

Test Case 18: Forgot Password with Wrong Email

This test falls under Exception Handling Testing. When a user clicks "Forgot Password" and enters an email address that does not exist in the database, the system detects this exception, shows an error message saying, "Email not found in our records" and does not proceed with password reset. This prevents unauthorized password reset attempts and informs the user to use a registered email or create a new account.

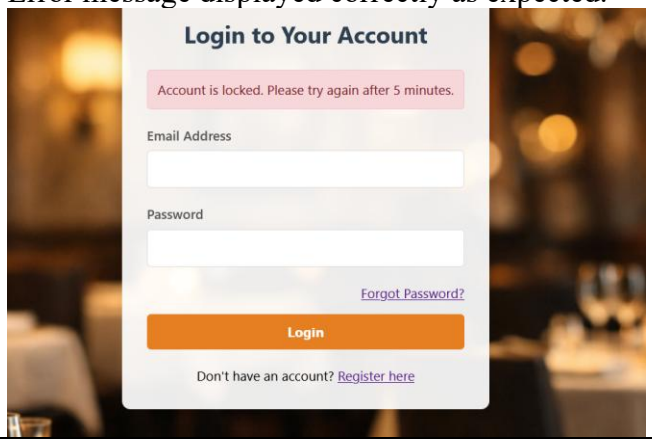
Table 18:Exception Handling testing (test18)

Test ID	18
Test Description	Verify that the system shows proper error message when user tries to reset password with an email not registered in the system.
Input Data	Email: wrongemail@test.com (not registered in database) 
Expected Output	Error message "Email not found."
Actual Output	Error message displayed correctly as expected. 

Test Case 19: Account Lock After Multiple Wrong Attempts

This test falls under Exception Handling Testing. It checks whether the system locks a user account after three consecutive wrong password attempts. When a user enters the wrong password three times, the system detects this as a potential brute force attack, locks the account and shows an error message saying "Account is locked. Please try again after 5 minutes." After 5 minutes, the account automatically unlocks and the user can try logging in again. This security feature prevents hackers from guessing passwords by trying many times.

Table 19: Account Lock After Multiple Wrong Attempts (test19)

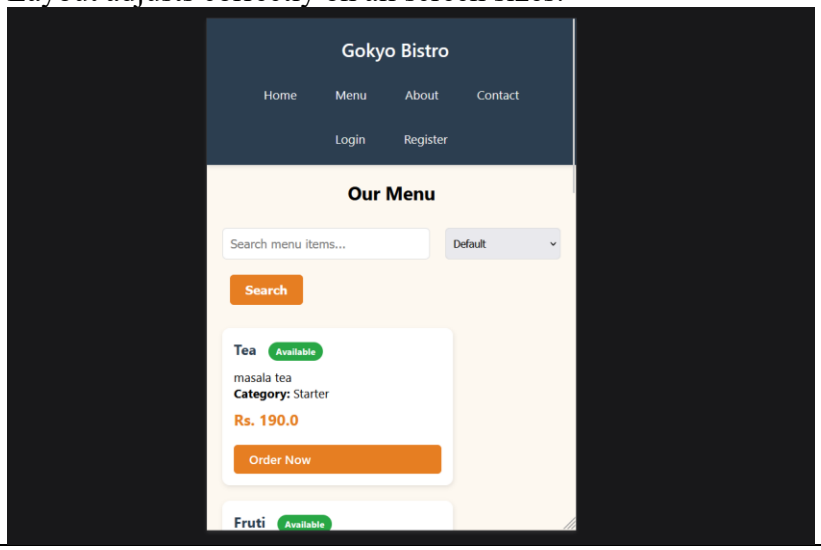
Test ID	19
Test Description	Verify that the system locks the account after 3 consecutive wrong password attempts and prevents login for 5 minutes.
Input Data	Correct email, wrong password entered 3 times in a row
Expected Output	After 3rd wrong attempt, account locked for 5 minutes. Error message "Account is locked. Please try again after 5 minutes."
Actual Output	Error message displayed correctly as expected. 

5.4 UI Testing

Test Case 20: Menu Page Responsive Layout

This test falls under UI Testing. It checks whether the menu page adjusts properly on different screen sizes. When viewed on a desktop, menu items display in 3 cards per row. On a tablet, they display in 2 cards per row. On a mobile phone, they display in 1 card per row. This ensures customers can view the menu comfortably on any device whether they are using a computer, tablet or mobile phone.

Table 20: Menu Page Responsive Layout(test 20)

Test ID	20
Test Description	Verify that menu items display in proper grid layout on desktop and mobile screens.
Input Data	Open menu page on different screen sizes (desktop, tablet, mobile)
Expected Output	Desktop shows 3 cards per row, mobile shows 1 card per row.
Actual Output	Layout adjusts correctly on all screen sizes. 

6. Coursework Development

This section describes the entire development process of the Gokyo Bistro Restaurant Management System. I will explain all the tools I used, how I used them and what I created with each tool. I will also explain my database design and how all the tables are connected.

6.1 Tools Used for Development

I used several different tools to build this project. Each tool helped me complete a specific part of the work. Some tools were for writing code, some for running the application, some for managing the database and some for creating diagrams and charts.

Tool 1: Eclipse IDE

Tool Description:

→ Eclipse IDE is a software where developers write Java code. It is free and widely used for building web applications. Eclipse helps in organizing code files, finding errors and running programs. It has many features that make coding easier, like automatic code completion, syntax highlighting and debugging tools.

Usage:

I used Eclipse to write all the Java code for my Gokyo Bistro project. I created servlets to handle user requests like login, registration, table booking and order placement. I created JSP pages for the user interface such as login page, register page, menu page, dashboard pages and many more. I also created model classes to represent data like UserModel, MenuModel, ReservationModel and OrderModel. I created service classes to handle business logic like UserService, MenuService, ReservationService and OrderService. I also created utility classes for password encryption and image upload. Eclipse helped me manage all these files in one place.

I also used Eclipse to run my project on Tomcat server. I could see any errors immediately as I typed, which helped me fix problems quickly. The debugger in Eclipse helped me find issues when my code was not working as expected. I could set breakpoints and see variable values step by step.

Evidence:

The screenshot below shows my Eclipse workspace with the GokyoBistroSystem project open. You can see all the packages like controllers, model, service and util. You can also see all the JSP files inside the WEB-INF folder.

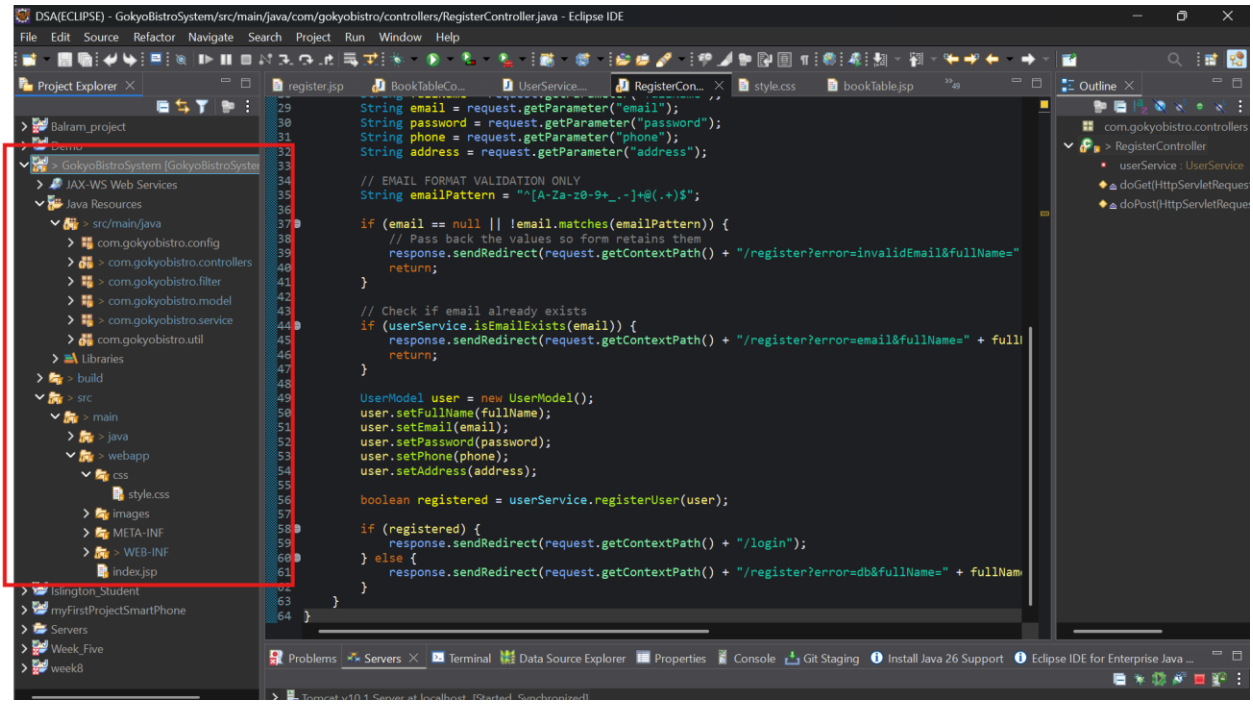


Figure 14:Evidence showing Eclipse IDE used in project

Tool 2: Apache Tomcat

Tool Description:

Apache Tomcat is a web server that runs Java servlets and JSP pages. When someone types a URL in their browser, Tomcat receives that request, processes it using my Java code and sends back the HTML response. Tomcat is free and works well with Eclipse.

Usage:

I used Tomcat version 10.1 to run my Gokyo Bistro application. I added Tomcat to Eclipse as a server. Then I deployed my project to this server. Every time I made changes to my code, I would restart Tomcat to see the changes. I tested all my features by opening my browser and going to http://localhost:8080/GokyoBistroSystem. Tomcat handled all requests for login, registration, menu display, table booking, order placement and admin functions.

I also used Tomcat logs to find errors. When something went wrong, I would look at the console in Eclipse to see what error message Tomcat printed. This helped me fix many bugs.

Evidence:

The screenshot below shows Tomcat running in the Servers tab of Eclipse. You can see that Tomcat has started and my project is deployed.

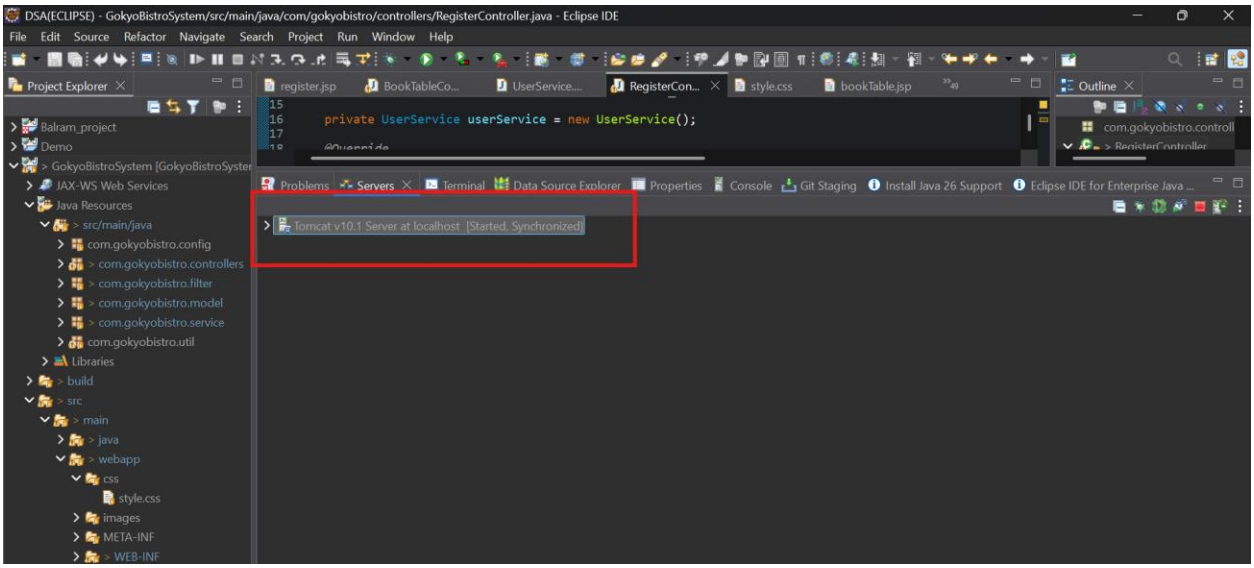


Figure 15:Apache Tomcat used in Eclipse.

Tool 3: MySQL and XAMPP

Tool Description:

MySQL is a database system that stores all the data for my application. It keeps information about users, menu items, reservations, orders. XAMPP is a software package that includes MySQL, Apache and phpMyAdmin. It makes it easy to start and stop the database server.

Usage:

I used XAMPP to start the MySQL server. I opened the XAMPP control panel and clicked the Start button for MySQL. This made the database available for my application to connect to. Then I opened phpMyAdmin by clicking the admin button. phpMyAdmin is a web-based tool that lets me create and manage databases without writing SQL commands manually.

In phpMyAdmin, I created a database called gokyo_bistro_db. Then I created four tables inside this database: user, menu, reservation, and orders. For each table, I defined the columns, data types, and constraints. I set primary keys on columns like user_id, menu_id, reservation_id and order_id. I also set foreign keys so that reservation.user_id references user.user_id, orders.user_id references user.user_id and orders.menu_id references menu.menu_id.

I also used phpMyAdmin to insert sample data into my tables so I could test my application. For example, I added an admin user and a few member users. I added several menu items like Momo, Chowmein, Pizza, and Beverages. I also added some sample reservations and orders to test the display pages.

Evidence:

The screenshot below shows the XAMPP control panel with MySQL running. The next screenshot shows my database gokyo_bistro_db with all four tables in phpMyAdmin. The next screenshot shows the structure of the user table with all its columns.

Figure 16: XAMPP Control Panel.

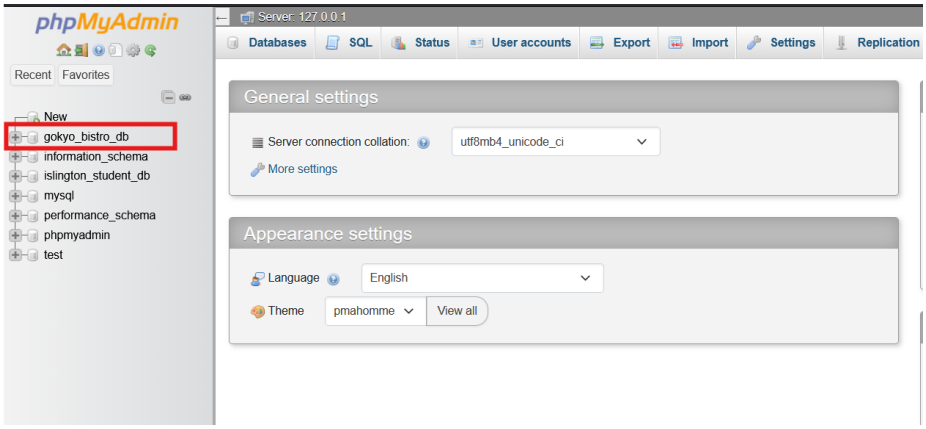


Figure 17: phpMyAdmin database

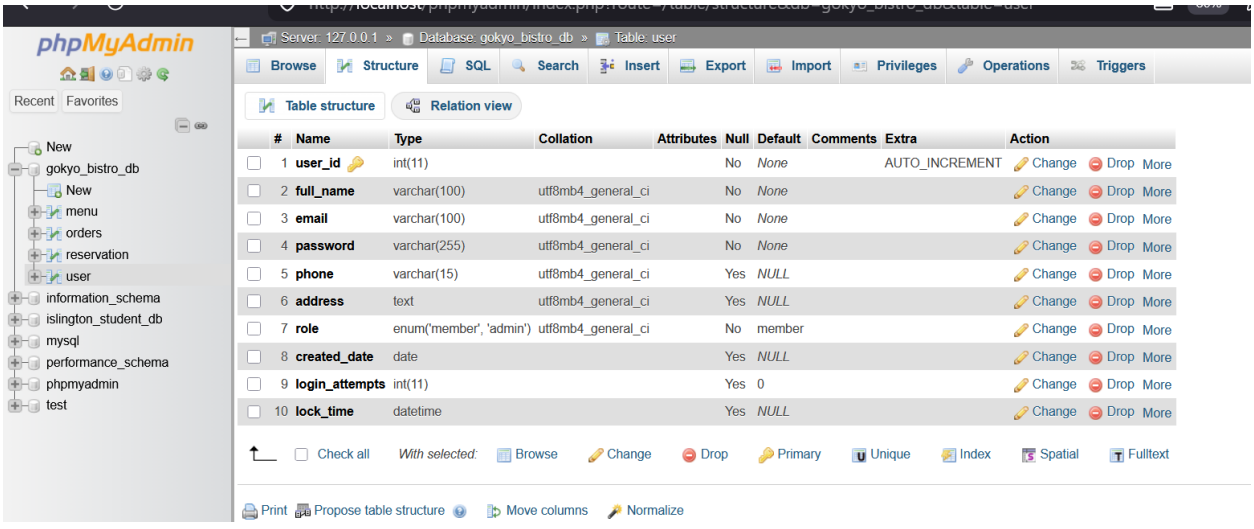


Figure 18: user table structure.

Tool 4: Git and GitHub

Tool Description:

Git is a version control system that tracks changes to code over time. It allows me to save different versions of my project and go back to an older version if something breaks. GitHub is a website where I can store my Git repositories online. It acts as a backup and allows me to share my code with others.

Usage:

I used Git to track all changes in my Gokyo Bistro project. Every time I completed a feature or fixed a bug, I committed my changes with a message describing what I did. I wrote commit messages like "Added login functionality", "Fixed table booking validation" or "Added forgot password feature". This helped me keep a history of my work.

I used GitHub to store my project online. I created a repository called Gokyo-Bistro- on GitHub. I linked my Eclipse project to this repository. Whenever I made changes, I pushed them to GitHub. This ensured that I would not lose my work even if something happened to my computer. It also made it easy to submit my coursework because I could just share the GitHub link.

The main purpose for uses for GitHub to give access to the module teacher access my code by providing my GitHub link in upper part.

Tool 5: Draw.io

Tool Description:

Draw.io is a free online tool for making diagrams. It has many shapes and symbols for UML diagrams, flowcharts, network diagrams and ER diagrams. It works in a web browser and does not require installation.

Usage:

I used Draw.io to create the class diagram for my coursework report. I created a class diagram to show all my model classes which are UserModel, MenuModel, ReservationModel and OrderModel. I also showed the relationships between these classes like how UserModel connects to ReservationModel and OrderModel, and how MenuModel connects to OrderModel.

To create the class diagram, I opened Draw.io, selected a blank diagram, and dragged rectangle shapes onto the canvas. I used three section rectangles for each class. The top section contains the class name. The middle section contains the attributes like userId, fullName, email, password. The bottom section contains the methods like getters and setters. Used straight line to show asscociation connection between model classes. I added multiplicity numbers like 1 and 0..* to show how many objects are connected. Then I exported the diagram as a PNG file and inserted it into my report.

Evidence:- Above class diagram is the evidence showing I used Draw.io in my coursework development.

6.2 Database Tables

I created four tables in my database named gokyo_bistro_db. Below I explain each table in detail.

Table 1: user

This table stores all the people who can use the system. This includes members who book tables and order food, and the admin who manages the restaurant.

The user_id column is the primary key. It is an integer that automatically increases whenever a new user is added. The full_name column stores the person's full name. The email column stores the login email address. This must be unique so that no two users can have the same email. The password column stores the encrypted password. I never store plain text passwords for security reasons. The phone column stores the user's phone number. The address column stores the user's address for delivery orders. The role column tells whether the user is a member or admin. The created_date column stores the date when the account was created. The login_attempts column counts how many times the user entered a wrong password. This is used for the account lock feature. The lock_time column stores the time when the account was locked. After three wrong attempts, the account locks for five minutes.

Table 2: menu

This table stores all the food and drink items that customers can order and displayed in Menu.

The menu_id column is the primary key. It auto increments for each new item. The item_name column stores the name of the food or drink like "Momo" or "Coke". The category column groups items into Starter, Main Course, Dessert or Beverage. This helps customers filter the menu. The description column gives more details about the item like what ingredients are used. The price column stores the cost in Nepali rupees. The availability column tells whether the item is currently available for ordering. Admin can set this to Yes or No. The created_date column stores when the item was added to the menu.

Table 3: reservation

This table stores all table bookings made by members.

The reservation_id column is the primary key. It auto-increments for each new booking. The user_id column stores which member made the booking. This is a foreign key that references the user_id column in the user table. The reservation_date column stores which date the customer wants to come. The reservation_time column stores what time they want to come, like 7:00 PM. The table_number column stores which table they booked. The restaurant has tables 1 to 6 with different seating capacities. The number_of_guests column stores how many people are coming. The status column tells if the booking is confirmed, cancelled or completed. The created_date column stores when the booking was made.

Table 4: orders

This table stores all delivery orders placed by members.

The order_id column is the primary key. It auto increments for each new order. The user_id column stores which member placed the order. This is a foreign key to the user table. The menu_id column stores which food item was ordered. This is a foreign key to the menu table. The quantity column stores how many of that item the customer wants. The total_price column stores the calculated cost, which is price multiplied by quantity. The delivery_address column stores where the food should be delivered. The status column tracks if the order is pending, confirmed, delivered or cancelled. The order_date column stores when the order was placed.

6.3 Entity Relationship Diagram

The diagram below shows how all four tables are connected.

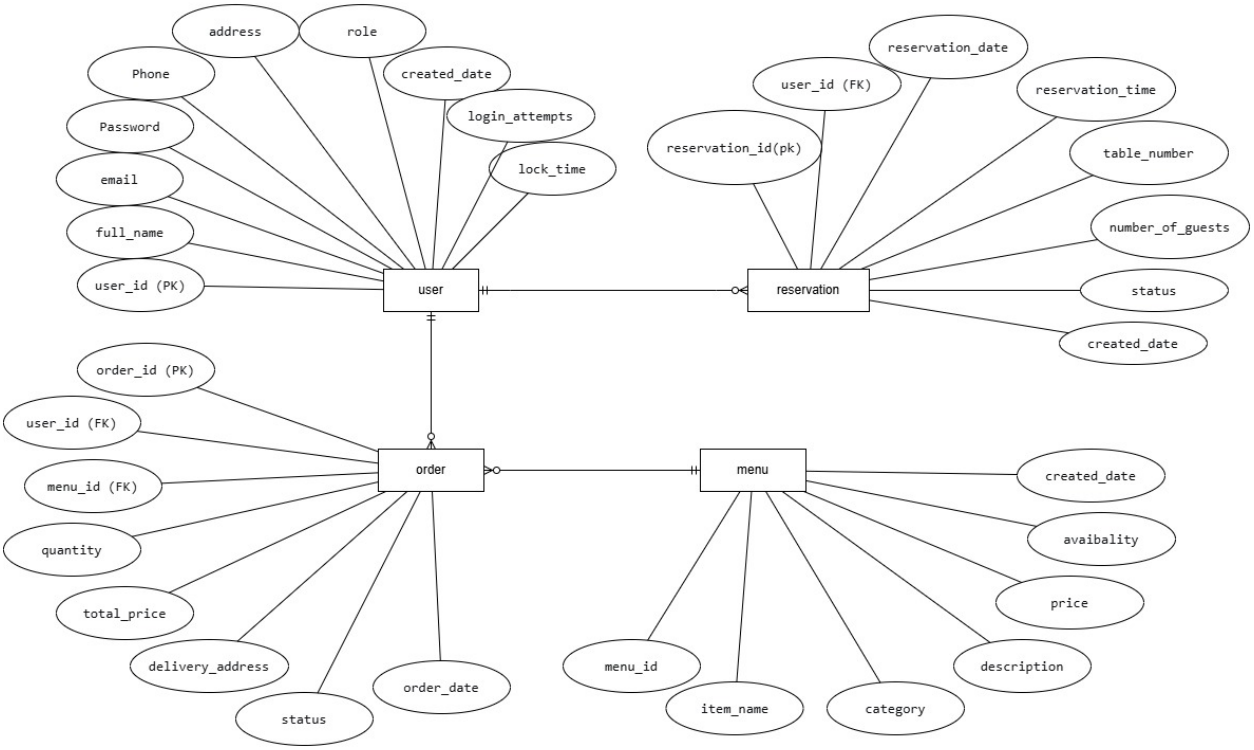


Figure 19:Database ER diagram

6.4 Database Relationships

There are three main relationships in my database.

The first relationship is between user and reservation. One user can make many reservations over time. For example, Amit can book a table for Friday and also book another table for Saturday. This is a one-to-many relationship. The reservation table has a foreign key called user_id that points to the user table.

The second relationship is between user and orders. One user can place many orders over time. Raghu can order Momo today and burger tomorrow. This is also a one-to-many relationship. The orders table has a foreign key called user_id that points to the user table.

The third relationship is between menu and orders. One menu item can appear in many orders. For example, many different customers can order pizza. This is a one-to-many relationship. The orders table has a foreign key called menu_id that points to the menu table.

Tool Used for Database

Tool: XAMPP with phpMyAdmin

Description: XAMPP is a free software package that includes Apache, MySQL, PHP and phpMyAdmin. I used it to run the MySQL database server. phpMyAdmin is a web-based tool that lets me manage my database through a graphical interface instead of typing SQL commands.

Usage: I opened XAMPP Control Panel and started the MySQL service. Then I clicked the Admin button, which opened phpMyAdmin in my browser. I clicked on New to create a new database and named it gokyo_bistro_db. Then I clicked on the SQL tab to run commands to create tables. I also used the Insert tab to add sample data. I used the Structure tab to add foreign keys. phpMyAdmin made it easy to see all my data and check if my tables were set up correctly.

7. Critical Analysis

I will discuss the problems I faced while building this system, what went well and what did not, how my system is better than manual systems and how fast the system works.

7.1 System Breakdown (Problems I Faced)

During the development of this project, I faced several technical problems. I will explain each problem and how I solved it.

Problem 1: Tomcat Server Would Not Start

One day when I opened Eclipse, Tomcat would not start. It showed an error saying "Port 8080 already in use". I did not know what this meant at first. I searched online and learned that another program was using the same port. I opened Command Prompt as administrator and typed netstat -ano | findstr :8080 to find the process using the port. Then I used taskkill /PID 4704 /F to stop that process. After this, Tomcat started normally.

Evidence: The screenshot below shows the port 8080 error message.

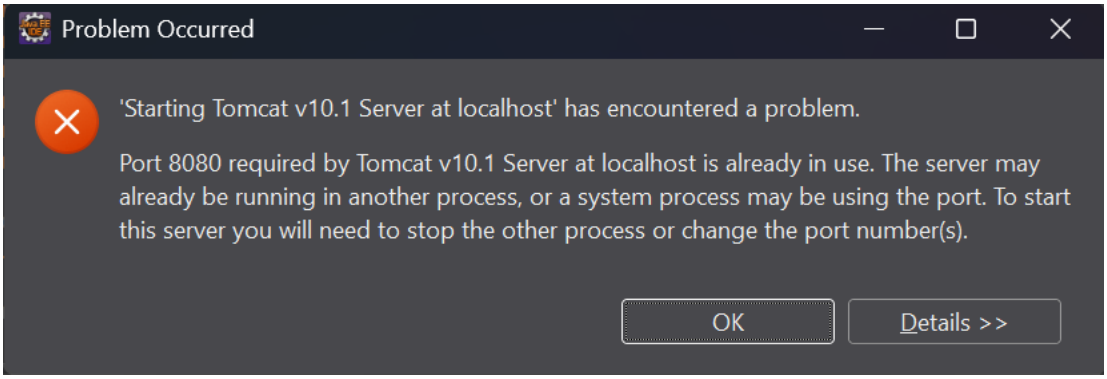


Figure 20: Port 8080 error message

Problem 2: MySQL Foreign Key Constraint Error

When I tried to delete a member from the admin panel, I got an error saying "Cannot delete or update a parent row: a foreign key constraint fails". This happened because the member had orders in the orders table. The database would not let me delete the user while orders still existed.

I solved this by creating a method called deleteMemberWithAllData(). This method first deletes all orders of that member, then deletes all reservations and finally deletes the user account. This way all related data is removed properly and no error occurs.

Problem 3: Menu Items Would Not Show in Grid Layout

After searching for menu items, the results were shown in a vertical list instead of the grid layout. This looked bad and was not consistent with the normal menu display.

I fixed this by checking my CSS. The problem was that the search results page was not using the same menu-grid CSS class. I made sure that both normal menu and search results use the same div class, so they look the same.

Problem 4: Update Menu Item Was Not Working

When admin tried to update a menu item, nothing happened. The page would just refresh and the changes were not saved.

I added debug print statements to see what was happening. I found that the menuId parameter was not being passed correctly from the edit form. The hidden input field for menuId was missing. I added <input type="hidden" name="menuId" value="<%= item.getMenuId() %>"> to the edit form. After this, the update worked correctly.

Problem 5: Account Lock Feature Not Working

I wanted to lock user accounts after three wrong password attempts. At first, this was not working. The login attempts were not being counted.

I checked my database and found that the login_attempts and lock_time columns were missing. I added these columns using SQL commands. Then I updated my login() method in UserService to increment the attempts on wrong password and lock the account after three attempts. After 5 minutes, the account unlocks automatically. This feature now works properly.

Problem 6: Email Validation Not Showing Error Message

When a user entered an invalid email during registration, the error message was not showing on the page. The URL showed ?error=invalidEmail but the red box did not appear.

I checked my JSP and found that the error condition was after other conditions. I moved the invalidEmail check to the top. I also used inline styles to make sure the error box appears. After this change, the error message shows correctly.

7.2 Evaluation - SWOT Analysis

SWOT stands for Strengths, Weaknesses, Opportunities and Threats. This is a way to evaluate my system.

Strengths (What is good in my system)

My system has several strong points. First, it has role based access control. This means members and admin see different pages. Admin can manage menu and members, while members can only book tables and order food. Second, the system has a table availability check. When a member tries to book a table that is already taken, the system shows an error message. Third, the system has an account lock feature. If someone tries wrong password three times, the account locks for 5 minutes. This stops hackers from guessing passwords. Fourth, the system is responsive. It works on mobile phones, tablets and computers. Fifth, the system encrypts passwords. No plain text passwords are stored in the database.

Weaknesses (What needs improvement in my system)

My system also has some weak points. First, there is no email notification. When a user registers or books a table, they do not get a confirmation email. Second, there is no payment gateway integration. The system shows a payment screen but does not actually process real payments.

Third, there is no search filter by category on the menu page. Users can only search by typing keywords. Fourth, the admin cannot edit their own profile. Only members can update their information. Fifth, there is no report export feature. Admin cannot download reports as PDF or Excel files.

Opportunities (What I can add in the future)

There are many opportunities to make my system better. First, I can add email notifications using JavaMail API. Users will get confirmation emails for registration, table booking and order placement. Second, I can integrate real payment gateways like Khalti or eSewa. This will allow customers to actually pay online. Third, I can add a barcode or QR code system for table booking. Customers can show the QR code at the restaurant for faster check-in. Fourth, I can add a mobile app version for Android and iOS. Fifth, I can add a loyalty points system. Regular customers can earn points and get discounts.

Threats (What can go wrong with my system)

There are some threats that can affect my system. First, database security is a threat. If someone gets access to the database, they can see user data. I have encrypted passwords, but emails and phone numbers are visible. Second, server downtime is a threat. If the hosting server goes down, customers cannot book tables. Third, competition is a threat. Other restaurants with better online systems may attract more customers. Fourth, cyber attacks like SQL injection and cross-site scripting are threats. I have used PreparedStatement to prevent SQL injection, but there is always risk.

7.3 Comparison with Other Existing Systems

Many restaurants still use manual systems for table booking. Customers must call the restaurant by phone to book a table. This has many problems.

Problems with existing manual systems

First, customers cannot see which tables are available. They have to call and ask. Second, double booking can happen. Two different customers can call at the same time and book the same table. Third, restaurant staff have to write down every booking on paper. This is slow and can lead to mistakes. Fourth, customers cannot see the menu online. They only know about the food when they come to the restaurant. Fifth, there is no order history. Customers cannot see what they ordered last time.

How my system solves these problems

My Gokyo Bistro system solves all these problems. First, customers can see real-time table availability on the website. They can see which tables are free at which time. Second, double booking is impossible because the system checks the database before confirming any booking. If a table is already booked, the system shows an error. Third, all bookings are stored in the database. No paperwork is needed. Fourth, customers can see the full menu online with prices and descriptions. Fifth, customers can see their order history in "My Orders" page. They can see what they ordered before and when.

Comparison with other online restaurant systems

There are other online restaurant systems like Foodmandu and Bhoj. But my system is customized for a single restaurant. Big platforms like Foodmandu list many restaurants, so they have to follow a standard template. My system is built specifically for Gokyo Bistro. It can be customized easily. Also, my system has no commission fees because it is owned by the restaurant. On Foodmandu, restaurants must pay commission on every order.

7.4 Performance Metrics

I tested the performance of my system to see how fast it works.

Search Performance

I tested the menu search feature. I searched for the keyword "momo". The search took less than 1 second to return results. I searched for "chicken". It also took less than 1 second. I searched for a random keyword like "xyz" that has no results. The search still completed in less than 1 second. The search is fast because the database table has indexes on the item_name column.

Page Load Time

I tested how long each page takes to load. The login page loads in about 0.5 seconds. The menu page takes about 0.8 seconds because it loads 6 to 8 menu items from the database. The admin dashboard loads in about 0.6 seconds. The booking page loads in about 0.4 seconds. The order page loads in about 0.5 seconds.

Database Query Performance

All database queries run quickly. The SELECT queries for getting menu items take about 0.1 to 0.2 seconds. The INSERT queries for booking a table or placing an order take about 0.1 seconds. The UPDATE queries for cancelling a reservation or updating profile take about 0.1 seconds. The DELETE query for removing a member takes about 0.2 seconds because it deletes from multiple tables.

What affects performance

The performance can slow down if there are many users at the same time. But for a single restaurant, this is not a big problem. The performance can also slow down if the database is not optimized. I have added indexes on important columns like email in the user table and item_name in the menu table to keep searches fast.

Performance on different devices

I tested the system on a laptop with Windows 11 and Chrome browser. Everything was fast. I also tested on a mobile phone using Chrome Developer Tools. The pages loaded in about 1 to 1.5 seconds on mobile. The responsive design made the pages look good on small screens.

8. Conclusion and Future Recommendation

8.1 Conclusion

I successfully built the Gokyo Bistro Restaurant Management System. The system allows members to book tables online, order food for delivery, cancel reservations, update their profile. The admin can manage menu items, view all reservations and orders and manage member accounts

During this project, I learned many new skills. I learned how to create a dynamic web application using Java, JSP, Servlets, and MySQL. I learned how to use MVC architecture to separate my code into model, view and controller layers. I learned how to handle user sessions and implement role based access control so that members and admin see different pages.

I faced several challenges while building this system. One challenge was setting up Tomcat and fixing the port 8080 error. I solved this by killing the process using that port. Another challenge was handling foreign key constraints when deleting members. I solved this by creating a method that deletes orders and reservations before deleting the user. I also faced challenges with form validation and keeping form data after errors. I solved this by passing data back through the URL and repopulating the form fields.

The system has some limitations. There is no email notification feature. Users do not get confirmation emails after registration or booking. There is no real payment gateway integration. The payment screen is simulated only. Admin cannot export reports as PDF or Excel files.

Despite these limitations, the system meets all the core requirements. Members can book tables and order food. Admin can manage the menu and members. The system is responsive and works on different screen sizes.

8.2 Future Recommendations

I have several ideas to improve this system in the future.

Add Email Notifications

I can add email notifications using JavaMail API. When a user registers, they will get a welcome email. When they book a table, they will get a confirmation email. When they place an order, they will get an order summary email. This will make the system more professional and keep customers informed.

Integrate Real Payment Gateway

I can integrate real payment gateways like Khalti or eSewa. Customers will be able to pay online using their mobile wallets or bank cards. This will make the ordering process complete and convenient.

Add Forgot Password with Email

Currently, users can reset password by entering email and new password directly. I can improve this by sending a reset link to the user's email. This is more secure and standard for websites.

Add Report Export Feature

I can add buttons to export reports as PDF or Excel files. Admin can download revenue reports and share them with restaurant management. This will help in business analysis.

Add Loyalty Points System

I can add a loyalty points system where members earn points for every order. They can redeem points for discounts on future orders. This will encourage customers to order more often.

Build Mobile App

I can build a mobile app for Android and iOS using the same backend. Customers can book tables and order food from their phones more easily.

These improvements will make the Gokyo Bistro system more complete and competitive with other restaurant management systems.

9. References

A) XAMPP download: - guidance provided by college lecture pdf in year while studying database module.

Download available at: https://www.apachefriends.org/download.html. Accessed on: november12 10, 2024.

B) Eclipse IDE for Enterprise Java download: - Guidance provided by lecture pdf in the second semester in year 2 in Data structure and specialist programming module.

Download Available at: https://www.eclipse.org/downloads/. Accessed on: March 9, 2026.

C) Apache Tomcat 10.1 Download: used for server purpose in eclipse

Download Available at: https://tomcat.apache.org/download-10.cgi. Accessed on: March 9, 2026

D) Used GeeksforGeeks for Java JDBC Tutorial.

Available at: https://www.geeksforgeeks.org/jdbc-tutorial/. Accessed on: April 10, 2026.

E) Used GeeksforGeeks for SHA-256 Hash in Java.

Available at: https://www.geeksforgeeks.org/sha-256-hash-in-java/. Accessed on: April 29, 2026.

F) Used Draw.io to prepare Class diagram and Er diagram

Available at: https://app.diagrams.net/.

H) W3Schools. SQL Tutorial. Available at: https://www.w3schools.com/sql/. Accessed on: April 11, 2026.

10. Appendices

```
package com.gokyobistro.service;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

import com.gokyobistro.config.DbConfig;
import com.gokyobistro.model.MenuModel;

/**
 * Handles menu-related business logic and database operations
 */
public class MenuService {

    /**
     * CREATE - Adds a new menu item to database
     */
    public boolean addMenuItem(MenuModel menu) {
        String sql = "INSERT INTO menu (item_name, category, description, price, "
                + "availability, created_date) VALUES (?, ?, ?, ?, ?, CURDATE())";

        try (Connection conn = DbConfig.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, menu.getItemName());
            pstmt.setString(2, menu.getCategory());
            pstmt.setString(3, menu.getDescription());
            pstmt.setDouble(4, menu.getPrice());
            pstmt.setString(5, menu.getAvailability());

            return pstmt.executeUpdate() > 0;

        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    /**
     * READ - Gets all menu items with sorting
     */
```

```
public List<MenuModel> getAllMenuItems(String sortBy) {
    List<MenuModel> menuList = new ArrayList<>();
    String sql = "SELECT * FROM menu " + getSortQuery(sortBy);

    try (Connection conn = DbConfig.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql);
         ResultSet rs = pstmt.executeQuery()) {

        while (rs.next()) {
            MenuModel menu = new MenuModel();
            menu.setMenuId(rs.getInt("menu_id"));
            menu.setItemName(rs.getString("item_name"));
            menu.setCategory(rs.getString("category"));
            menu.setDescription(rs.getString("description"));
            menu.setPrice(rs.getDouble("price"));
            menu.setAvailability(rs.getString("availability"));
            menu.setCreatedDate(rs.getString("created_date"));
            menuList.add(menu);
        }

    } catch (ClassNotFoundException | SQLException e) {
        e.printStackTrace();
    }

    return menuList;
}

/**
 * READ - Gets all menu items (without sorting - for backward compatibility)
 */
public List<MenuModel> getAllMenuItems() {
    return getAllMenuItems("default");
}

/**
 * READ - Gets menu items by category
 */
public List<MenuModel> getMenuItemsByCategory(String category) {
    List<MenuModel> menuList = new ArrayList<>();
    String sql = "SELECT * FROM menu WHERE category = ? AND availability = 'Yes'";

    try (Connection conn = DbConfig.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString(1, category);
        ResultSet rs = pstmt.executeQuery();
```

```
        while (rs.next()) {
            MenuModel menu = new MenuModel();
            menu.setMenuId(rs.getInt("menu_id"));
            menu.setItemName(rs.getString("item_name"));
            menu.setCategory(rs.getString("category"));
            menu.setDescription(rs.getString("description"));
            menu.setPrice(rs.getDouble("price"));
            menu.setAvailability(rs.getString("availability"));
            menu.setCreatedDate(rs.getString("created_date"));
            menuList.add(menu);
        }

    } catch (ClassNotFoundException | SQLException e) {
        e.printStackTrace();
    }

    return menuList;
}

/**
 * READ - Gets a single menu item by ID
 */
public MenuModel getMenuItemById(int menuId) {
    String sql = "SELECT * FROM menu WHERE menu_id = ?";

    try (Connection conn = DbConfig.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setInt(1, menuId);
        ResultSet rs = pstmt.executeQuery();

        if (rs.next()) {
            MenuModel menu = new MenuModel();
            menu.setMenuId(rs.getInt("menu_id"));
            menu.setItemName(rs.getString("item_name"));
            menu.setCategory(rs.getString("category"));
            menu.setDescription(rs.getString("description"));
            menu.setPrice(rs.getDouble("price"));
            menu.setAvailability(rs.getString("availability"));
            menu.setCreatedDate(rs.getString("created_date"));
            return menu;
        }

    } catch (ClassNotFoundException | SQLException e) {
        e.printStackTrace();
```

```
    }

    return null;
}

/**
 * UPDATE - Updates an existing menu item
 */
public boolean updateMenuItem(MenuModel menu) {
    String sql = "UPDATE menu SET item_name = ?, category = ?, description = ?, "
            + "price = ?, availability = ? WHERE menu_id = ?";

    try (Connection conn = DbConfig.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString(1, menu.getItemName());
        pstmt.setString(2, menu.getCategory());
        pstmt.setString(3, menu.getDescription());
        pstmt.setDouble(4, menu.getPrice());
        pstmt.setString(5, menu.getAvailability());
        pstmt.setInt(6, menu.getMenuId());

        return pstmt.executeUpdate() > 0;

    } catch (ClassNotFoundException | SQLException e) {
        e.printStackTrace();
        return false;
    }
}

/**
 * DELETE - Removes a menu item from database
 */
public boolean deleteMenuItem(int menuId) {
    String sql = "DELETE FROM menu WHERE menu_id = ?";

    try (Connection conn = DbConfig.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setInt(1, menuId);
        return pstmt.executeUpdate() > 0;

    } catch (ClassNotFoundException | SQLException e) {
        e.printStackTrace();
        return false;
    }
```

```
    }

    /**
     * Search menu items by name or category with sorting
     */
    public List<MenuModel> searchMenuItems(String keyword, String sortBy) {
        List<MenuModel> menuList = new ArrayList<>();
        String sql = "SELECT * FROM menu WHERE item_name LIKE ? OR category LIKE ? "
+ getSortQuery(sortBy);

        try (Connection conn = DbConfig.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            String searchPattern = "%" + keyword + "%";
            pstmt.setString(1, searchPattern);
            pstmt.setString(2, searchPattern);
            ResultSet rs = pstmt.executeQuery();

            while (rs.next()) {
                MenuModel menu = new MenuModel();
                menu.setMenuId(rs.getInt("menu_id"));
                menu.setItemName(rs.getString("item_name"));
                menu.setCategory(rs.getString("category"));
                menu.setDescription(rs.getString("description"));
                menu.setPrice(rs.getDouble("price"));
                menu.setAvailability(rs.getString("availability"));
                menu.setCreatedDate(rs.getString("created_date"));
                menuList.add(menu);
            }

        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
        }

        return menuList;
    }

    /**
     * Search menu items by name or category (without sorting - for backward compatibility)
     */
    public List<MenuModel> searchMenuItems(String keyword) {
        return searchMenuItems(keyword, "default");
    }

    /**
     * Helper method to get ORDER BY clause based on sort parameter
```

```
    */
    private String getSortQuery(String sortBy) {
        if (sortBy == null || sortBy.equals("default")) {
            return "ORDER BY menu_id DESC";
        } else if (sortBy.equals("price_asc")) {
            return "ORDER BY price ASC";
        } else if (sortBy.equals("price_desc")) {
            return "ORDER BY price DESC";
        } else if (sortBy.equals("name_asc")) {
            return "ORDER BY item_name ASC";
        }
        return "ORDER BY menu_id DESC";
    }
}


package com.gokyobistro.service;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

import com.gokyobistro.config.DbConfig;
import com.gokyobistro.model.OrderModel;

/**
 Handles order related business logic and database operations

 */
public class OrderService {

    /*
      CREATE - Places a new food order

     */
    public boolean placeOrder(OrderModel order) {
        String sql = "INSERT INTO orders (user_id, menu_id, quantity, total_price, "
                + "delivery_address, status, order_date) "
                + "VALUES (?, ?, ?, ?, ?, 'pending', CURDATE())";

        try (Connection conn = DbConfig.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
```

```
            pstmt.setInt(1, order.getUserId());
            pstmt.setInt(2, order.getMenuId());
            pstmt.setInt(3, order.getQuantity());
            pstmt.setDouble(4, order.getTotalPrice());
            pstmt.setString(5, order.getDeliveryAddress());

            return pstmt.executeUpdate() > 0;

        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    /**
     * READ - Gets all orders for a specific user
     * userId User ID
     *  List of user's orders
     */
    public List<OrderModel> getOrdersByUser(int userId) {
        List<OrderModel> orders = new ArrayList<>();
        String sql = "SELECT o.*, u.full_name as user_name, m.item_name as menu_item_name
"
                + "FROM orders o "
                + "JOIN user u ON o.user_id = u.user_id "
                + "JOIN menu m ON o.menu_id = m.menu_id "
                + "WHERE o.user_id = ? ORDER BY o.order_date DESC";

        try (Connection conn = DbConfig.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setInt(1, userId);
            ResultSet rs = pstmt.executeQuery();

            while (rs.next()) {
                OrderModel order = new OrderModel();
                order.setOrderId(rs.getInt("order_id"));
                order.setUserId(rs.getInt("user_id"));
                order.setUserName(rs.getString("user_name"));
                order.setMenuId(rs.getInt("menu_id"));
                order.setMenuItemName(rs.getString("menu_item_name"));
                order.setQuantity(rs.getInt("quantity"));
                order.setTotalPrice(rs.getDouble("total_price"));
                order.setDeliveryAddress(rs.getString("delivery_address"));
                order.setStatus(rs.getString("status"));
                order.setOrderDate(rs.getString("order_date"));
```

```
            orders.add(order);
        }

    } catch (ClassNotFoundException | SQLException e) {
        e.printStackTrace();
    }

    return orders;
}

/**
 * READ Gets all orders (for Admin)
 *
 * List of all orders
 */
public List<OrderModel> getAllOrders() {
    List<OrderModel> orders = new ArrayList<>();
    String sql = "SELECT o.*, u.full_name as user_name, m.item_name as menu_item_name
"
            + "FROM orders o "
            + "JOIN user u ON o.user_id = u.user_id "
            + "JOIN menu m ON o.menu_id = m.menu_id "
            + "ORDER BY o.order_date DESC";

    try (Connection conn = DbConfig.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql);
         ResultSet rs = pstmt.executeQuery()) {

        while (rs.next()) {
            OrderModel order = new OrderModel();
            order.setOrderId(rs.getInt("order_id"));
            order.setUserId(rs.getInt("user_id"));
            order.setUserName(rs.getString("user_name"));
            order.setMenuId(rs.getInt("menu_id"));
            order.setMenuItemName(rs.getString("menu_item_name"));
            order.setQuantity(rs.getInt("quantity"));
            order.setTotalPrice(rs.getDouble("total_price"));
            order.setDeliveryAddress(rs.getString("delivery_address"));
            order.setStatus(rs.getString("status"));
            order.setOrderDate(rs.getString("order_date"));
            orders.add(order);
        }

    } catch (ClassNotFoundException | SQLException e) {
        e.printStackTrace();
    }
```

```
        return orders;
    }

    /**
     * UPDATE - Updates order status
      orderId Order ID
      status New status (confirmed, delivered, cancelled)
      true if successful, false otherwise
     */
    public boolean updateOrderStatus(int orderId, String status) {
        String sql = "UPDATE orders SET status = ? WHERE order_id = ?";

        try (Connection conn = DbConfig.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, status);
            pstmt.setInt(2, orderId);
            return pstmt.executeUpdate() > 0;

        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
            return false;
        }
    }
}

package com.gokyobistro.service;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

import com.gokyobistro.config.DbConfig;
import com.gokyobistro.model.ReservationModel;

/**
 Handles reservation related business logic and database operations

 */
public class ReservationService {

    /**
```

```
 * CREATE - Books a new table reservation
 reservation ReservationModel with booking details
 true if successful, false otherwise
 */
public boolean bookTable(ReservationModel reservation) {
    String sql = "INSERT INTO reservation (user_id, reservation_date, reservation_time, "
            + "table_number, number_of_guests, status, created_date) "
            + "VALUES (?, ?, ?, ?, ?, 'confirmed', CURDATE())";

    try (Connection conn = DbConfig.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setInt(1, reservation.getUserId());
        pstmt.setString(2, reservation.getReservationDate());
        pstmt.setString(3, reservation.getReservationTime());
        pstmt.setInt(4, reservation.getTableNumber());
        pstmt.setInt(5, reservation.getNumberOfGuests());

        return pstmt.executeUpdate() > 0;

    } catch (ClassNotFoundException | SQLException e) {
        e.printStackTrace();
        return false;
    }
}

/*
  READ - Gets all reservations for a specific user

 */
public List<ReservationModel> getReservationsByUser(int userId) {
    List<ReservationModel> reservations = new ArrayList<>();
    String sql = "SELECT r.*, u.full_name as user_name FROM reservation r "
            + "JOIN user u ON r.user_id = u.user_id "
            + "WHERE r.user_id = ? ORDER BY r.reservation_date DESC";

    try (Connection conn = DbConfig.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setInt(1, userId);
        ResultSet rs = pstmt.executeQuery();

        while (rs.next()) {
            ReservationModel reservation = new ReservationModel();
            reservation.setReservationId(rs.getInt("reservation_id"));
            reservation.setUserId(rs.getInt("user_id"));
```

```
            reservation.setUserName(rs.getString("user_name"));
            reservation.setReservationDate(rs.getString("reservation_date"));
            reservation.setReservationTime(rs.getString("reservation_time"));
            reservation.setTableNumber(rs.getInt("table_number"));
            reservation.setNumberOfGuests(rs.getInt("number_of_guests"));
            reservation.setStatus(rs.getString("status"));
            reservation.setCreatedDate(rs.getString("created_date"));
            reservations.add(reservation);
        }

    } catch (ClassNotFoundException | SQLException e) {
        e.printStackTrace();
    }

    return reservations;
}

/**
 * READ  Gets all reservations (for Admin)
 List of all reservations
 */
public List<ReservationModel> getAllReservations() {
    List<ReservationModel> reservations = new ArrayList<>();
    String sql = "SELECT r.*, u.full_name as user_name FROM reservation r "
            + "JOIN user u ON r.user_id = u.user_id "
            + "ORDER BY r.reservation_date DESC";

    try (Connection conn = DbConfig.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql);
         ResultSet rs = pstmt.executeQuery()) {

        while (rs.next()) {
            ReservationModel reservation = new ReservationModel();
            reservation.setReservationId(rs.getInt("reservation_id"));
            reservation.setUserId(rs.getInt("user_id"));
            reservation.setUserName(rs.getString("user_name"));
            reservation.setReservationDate(rs.getString("reservation_date"));
            reservation.setReservationTime(rs.getString("reservation_time"));
            reservation.setTableNumber(rs.getInt("table_number"));
            reservation.setNumberOfGuests(rs.getInt("number_of_guests"));
            reservation.setStatus(rs.getString("status"));
            reservation.setCreatedDate(rs.getString("created_date"));
            reservations.add(reservation);
        }

    } catch (ClassNotFoundException | SQLException e) {
```

```
            e.printStackTrace();
        }

        return reservations;
    }

    /**
     * UPDATE - Cancels a reservation
      reservationId Reservation ID to cancel
     true if successful, false otherwise
     */
    public boolean cancelReservation(int reservationId) {
        String sql = "UPDATE reservation SET status = 'cancelled' WHERE reservation_id = ?";

        try (Connection conn = DbConfig.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setInt(1, reservationId);
            return pstmt.executeUpdate() > 0;

        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    /**
     * Checks if table is available at given date and time
      tableNumber Table number
      date Reservation date
      time Reservation time
      true if available, false otherwise
     */
    public boolean isTableAvailable(int tableNumber, String date, String time) {
        String sql = "SELECT COUNT(*) FROM reservation WHERE table_number = ? "
                + "AND reservation_date = ? AND reservation_time = ? "
                + "AND status = 'confirmed'";

        try (Connection conn = DbConfig.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setInt(1, tableNumber);
            pstmt.setString(2, date);
            pstmt.setString(3, time);
            ResultSet rs = pstmt.executeQuery();
```

```
            if (rs.next()) {
                return rs.getInt(1) == 0;
            }

        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
        }

        return true;
    }
}


package com.gokyobistro.service;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Timestamp;
import java.util.ArrayList;
import java.util.List;
import java.security.NoSuchAlgorithmException;

import com.gokyobistro.config.DbConfig;
import com.gokyobistro.model.UserModel;
import com.gokyobistro.util.PasswordUtil;

public class UserService {

    // REGISTER USER
    public boolean registerUser(UserModel user) {
        String sql = "INSERT INTO user (full_name, email, password, phone, address, role,
created_date) "
                + "VALUES (?, ?, ?, ?, ?, ?, CURDATE())";

        try (Connection conn = DbConfig.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, user.getFullName());
            pstmt.setString(2, user.getEmail());
            String encryptedPassword = PasswordUtil.encryptPassword(user.getPassword());
            pstmt.setString(3, encryptedPassword);
            pstmt.setString(4, user.getPhone());
            pstmt.setString(5, user.getAddress());
            pstmt.setString(6, "member");
```

```
            return pstmt.executeUpdate() > 0;

        } catch (ClassNotFoundException | SQLException | NoSuchAlgorithmException e) {
            e.printStackTrace();
            return false;
        }
    }

    //LOGIN USER WITH ACCOUNT LOCK
    public UserModel login(String email, String password) {
        String sql = "SELECT * FROM user WHERE email = ?";

        try (Connection conn = DbConfig.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, email);
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                // Check if account is locked
                Timestamp lockTime = rs.getTimestamp("lock_time");
                if (lockTime != null) {
                    long diff = System.currentTimeMillis() - lockTime.getTime();
                    if (diff < 5 * 60 * 1000) { // Less than 5 minutes
                        System.out.println("Account locked. Please try after 5 minutes.");
                        return null;
                    } else {
                        // Reset lock after 5 minutes
                        resetLoginAttempts(email);
                    }
                }

                String storedPassword = rs.getString("password");

                if (PasswordUtil.verifyPassword(password, storedPassword)) {
                    // Login success - reset attempts
                    resetLoginAttempts(email);

                    UserModel user = new UserModel();
                    user.setUserId(rs.getInt("user_id"));
                    user.setFullName(rs.getString("full_name"));
                    user.setEmail(rs.getString("email"));
                    user.setPhone(rs.getString("phone"));
                    user.setAddress(rs.getString("address"));
                    user.setRole(rs.getString("role"));
```

```
                user.setCreatedDate(rs.getString("created_date"));
                return user;
            } else {
                // Wrong password - increment attempts
                incrementLoginAttempts(email);
                return null;
            }
        }

    } catch (ClassNotFoundException | SQLException | NoSuchAlgorithmException e) {
        e.printStackTrace();
    }
    return null;
}

// RESET LOGIN ATTEMPTS
private void resetLoginAttempts(String email) {
    String sql = "UPDATE user SET login_attempts = 0, lock_time = NULL WHERE email =
?";
    try (Connection conn = DbConfig.getConnection();
         PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, email);
        ps.executeUpdate();
    } catch (ClassNotFoundException | SQLException e) {
        e.printStackTrace();
    }
}

// COUNT LOGIN ATTEMPTS
private void incrementLoginAttempts(String email) {
    String sql = "UPDATE user SET login_attempts = login_attempts + 1 WHERE email = ?";
    try (Connection conn = DbConfig.getConnection();
         PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, email);
        ps.executeUpdate();

        // Check if attempts reached 3
        String selectSql = "SELECT login_attempts FROM user WHERE email = ?";
        PreparedStatement ps2 = conn.prepareStatement(selectSql);
        ps2.setString(1, email);
        ResultSet rs = ps2.executeQuery();
        if (rs.next() && rs.getInt(1) >= 3) {
            lockAccount(email);
        }
    } catch (ClassNotFoundException | SQLException e) {
        e.printStackTrace();
```

```
        }
    }

    // LOCK ACCOUNT
    private void lockAccount(String email) {
        String sql = "UPDATE user SET lock_time = NOW() WHERE email = ?";
        try (Connection conn = DbConfig.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, email);
            ps.executeUpdate();
        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
        }
    }

    //CHECK IF ACCOUNT IS LOCKED
    public boolean isAccountLocked(String email) {
        String sql = "SELECT lock_time FROM user WHERE email = ? AND lock_time IS NOT
NULL";
        try (Connection conn = DbConfig.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, email);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                Timestamp lockTime = rs.getTimestamp("lock_time");
                long diff = System.currentTimeMillis() - lockTime.getTime();
                return diff < 5 * 60 * 1000;
            }
        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
        }
        return false;
    }

    // CHECK IF EMAIL EXISTS
    public boolean isEmailExists(String email) {
        String sql = "SELECT COUNT(*) FROM user WHERE email = ?";

        try (Connection conn = DbConfig.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, email);
            ResultSet rs = pstmt.executeQuery();
            if (rs.next()) {
                return rs.getInt(1) > 0;
            }
```

```
        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
        }
        return false;
    }

    // GET USER BY ID
    public UserModel getUserById(int userId) {
        String sql = "SELECT * FROM user WHERE user_id = ?";

        try (Connection conn = DbConfig.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setInt(1, userId);
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                UserModel user = new UserModel();
                user.setUserId(rs.getInt("user_id"));
                user.setFullName(rs.getString("full_name"));
                user.setEmail(rs.getString("email"));
                user.setPhone(rs.getString("phone"));
                user.setAddress(rs.getString("address"));
                user.setRole(rs.getString("role"));
                user.setCreatedDate(rs.getString("created_date"));
                return user;
            }
        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
        }
        return null;
    }

    // UPDATE USER PROFILE
    public boolean updateUserProfile(UserModel user) {
        String sql;
        if (user.getPassword() != null && !user.getPassword().isEmpty()) {
            sql = "UPDATE user SET full_name = ?, phone = ?, address = ?, password = ? WHERE
user_id = ?";
        } else {
            sql = "UPDATE user SET full_name = ?, phone = ?, address = ? WHERE user_id = ?";
        }

        try (Connection conn = DbConfig.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
```

```
            pstmt.setString(1, user.getFullName());
            pstmt.setString(2, user.getPhone());
            pstmt.setString(3, user.getAddress());

            if (user.getPassword() != null && !user.getPassword().isEmpty()) {
                String encryptedPassword = PasswordUtil.encryptPassword(user.getPassword());
                pstmt.setString(4, encryptedPassword);
                pstmt.setInt(5, user.getUserId());
            } else {
                pstmt.setInt(4, user.getUserId());
            }

            return pstmt.executeUpdate() > 0;

        } catch (ClassNotFoundException | SQLException | NoSuchAlgorithmException e) {
            e.printStackTrace();
            return false;
        }
    }

    // GET ALL MEMBERS
    public List<UserModel> getAllMembers() {
        List<UserModel> membersList = new ArrayList<>();
        String sql = "SELECT * FROM user WHERE role = 'member' ORDER BY user_id
DESC";

        try (Connection conn = DbConfig.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql);
             ResultSet rs = pstmt.executeQuery()) {

            while (rs.next()) {
                UserModel user = new UserModel();
                user.setUserId(rs.getInt("user_id"));
                user.setFullName(rs.getString("full_name"));
                user.setEmail(rs.getString("email"));
                user.setPhone(rs.getString("phone"));
                user.setAddress(rs.getString("address"));
                user.setRole(rs.getString("role"));
                user.setCreatedDate(rs.getString("created_date"));
                membersList.add(user);
            }
        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
        }
        return membersList;
    }
```

```
// UPDATE PASSWORD BY EMAIL
public boolean updatePasswordByEmail(String email, String newPassword) {
    String sql = "UPDATE user SET password = ? WHERE email = ?";
    try (Connection conn = DbConfig.getConnection();
         PreparedStatement ps = conn.prepareStatement(sql)) {
        String encrypted = PasswordUtil.encryptPassword(newPassword);
        ps.setString(1, encrypted);
        ps.setString(2, email);
        return ps.executeUpdate() > 0;
    } catch (Exception e) {
        e.printStackTrace();
        return false;
    }
}
public boolean deleteMemberWithAllData(int userId) {
    try (Connection conn = DbConfig.getConnection()) {
        conn.setAutoCommit(false);

        // Delete orders
        String sql1 = "DELETE FROM orders WHERE user_id = ?";
        PreparedStatement ps1 = conn.prepareStatement(sql1);
        ps1.setInt(1, userId);
        ps1.executeUpdate();
        ps1.close();

        // Delete reservations
        String sql2 = "DELETE FROM reservation WHERE user_id = ?";
        PreparedStatement ps2 = conn.prepareStatement(sql2);
        ps2.setInt(1, userId);
        ps2.executeUpdate();
        ps2.close();

        // Delete user
        String sql3 = "DELETE FROM user WHERE user_id = ?";
        PreparedStatement ps3 = conn.prepareStatement(sql3);
        ps3.setInt(1, userId);
        int result = ps3.executeUpdate();

        conn.commit();
        return result > 0;

    } catch (Exception e) {
        e.printStackTrace();
        return false;
    }
```

```
    }
}
```